

侏罗纪网络安全加密套件（JPSSL）使用说明

JPSSL Wiki 合并版 · C++20 高性能跨平台密码学库

著作权人：侏罗纪（海城）计算机软件科技有限公司

版本号：1.1.9

开源地址：<https://github.com/DaChengTechnology/JPSSL>
https://gitee.com/dachtech_admin/JPSSL

生成日期：2026年08月16日

目录

JPSSL 维基	8
特性一览	8
开始使用	8
API 参考	8
进阶主题	9
相关链接	9
新手引导	10
JPSSL 是什么?	10
学习路线	10
环境要求	10
第 1 步: 获取代码	10
第 2 步: 构建	11
Linux / macOS	11
Windows (MSVC)	11
第 3 步: 运行测试 (可选但推荐)	11
第 4 步: 运行你的第一个程序	11
编译运行	12
集成到 CMake 项目 (推荐)	12
第 5 步: 用命令行工具快速体验	12
常见问题	12
下一步	13
快速开始	14
环境要求	14
Linux / macOS 构建	14

Windows 构建 (MSVC)	14
链接到你的项目	15
下一步	15
构建选项与产物	16
CMake 选项	16
构建产物	16
平台与编译器注意事项	17
Windows (MSVC)	17
Linux (GCC / Clang)	17
条件编译与运行时调度	17
依赖	18
移动端 / 桌面端平台构建	18
平台构建 (iOS / macOS / Android / HarmonyOS)	19
iOS (XCFramework, arm64)	19
macOS (arm64: 动态库 + framework)	19
Android (AAR, arm64-v8a)	19
HarmonyOS / OpenHarmony (arm64-v8a .a / .so)	20
命令行工具	21
jpssl-cert — 证书工具	21
jpssl-crypt — 加密 / 哈希工具	22
算法支持总览	23
TLS 密码套件	24
TLS 1.3	24
TLS 1.2 (RFC 5246)	24
标准合规	24
对称加密 API	26

AES (FIPS 197) 26

GHASH / GF(2^128) (GCM 认证核心, 见 NIST SP 800-38D, 可单独使用) 26

ChaCha20-Poly1305 (RFC 8439) 26

SM4 (GM/T 0002-2012) 27

SM4-GCM AEAD (NIST SP 800-38D 框架, SM4 见 GM/T 0002) 27

SM4-CCM AEAD (NIST SP 800-38C 框架, SM4 见 GM/T 0002) 28

Base64 (RFC 4648) 28

哈希 / MAC / KDF API 29

SHA-1 (FIPS 180-4) 29

SHA-256 (FIPS 180-4) 29

SHA-384 / SHA-512 (FIPS 180-4) 29

SHA3-256 / 384 / 512 (FIPS 202) 29

SM3 (GM/T 0004-2012) 30

HMAC (RFC 2104 / FIPS 198-1) 30

HKDF (RFC 5869) 30

随机数 30

非对称加密 API 31

RSA 2048 / 4096 (PKCS#1 v2.2 / RFC 8017) 31

ECDSA P-256 (FIPS 186-4) 31

Ed25519 (RFC 8032) 32

Ed448 (RFC 8032) 32

X25519 (RFC 7748) 32

X448 (RFC 7748) 32

SM2 (GM/T 0003-2012, RFC 8998 国密 TLS) 33

密钥与签名互操作 33

X.509 v3 证书 API (RFC 5280) 34

1. 生成自签名证书	34
2. 解析 DER / PEM 证书	34
3. 读取私钥 / CSR (PEM / DER)	35
4. 证书链验证	35
OpenSSL 互操作	35
5. TLS 集成	36
支持的关键类型	36
命令行工具	36
TLS API (TLS 1.2 / 1.3)	37
1. 创建证书	37
2. 多域名证书管理 (SNI)	37
3. TLS 1.3 完整握手 (推荐)	38
4. TLS 1.3 简化版握手 (一次性)	38
5. TLS 1.3 0-RTT (零往返) + PSK 会话恢复	39
6. TLS 1.2 握手 (RFC 5246)	40
7. 记录层加解密 (握手完成后)	40
8. 大消息自动分片与合并	41
9. 握手 API 总览	41
10. 证书管理 API	42
11. 会话状态	43
TLS socket 封装层	44
服务端示例	44
客户端示例	44
成员一览	44
外部 fd 托管	46
UDP 链接 (数据报模式) — ❷ ❷ 非标准, 存在缺陷, 仅限自研两端互通	46

- 已知缺陷（与标准 DTLS / QUIC 不互通） 46
- 非阻塞模式（事件循环） 47
- 协程 I/O（C++20） 47
- 客户端验证 48
- 相关示例 48
- 证书透明（Certificate Transparency） 49**
 - 核心概念与功能 49
 - 默克尔树 49
 - TLS 风格编解码 49
 - 内存日志 49
 - 签发与验证 50
 - 注意 50
 - HTTPS + CT 示例 50
- MUSA GPU 加速（实验性） 51**
 - 启用 51
 - GPU 内核 51
 - 持久化池 API 51
 - 宏守卫 51
 - 性能数据 52
- 性能基准 53**
 - 基准目标 53
 - 已公布的实测数据 53
 - 平台：MTT S80（GPU）+ i7-13700K（CPU） 53
 - 对 OpenSSL 的对比结论（来自提交历史 / README） 53
 - 说明 54
- 测试与 CI 55**

运行测试	55
单元测试目标	55
DTLS 1.2 / 1.3 合规性测试	56
TLS 稳定性测试	56
主测试程序	57
持续集成	57
ECDSA AVX2 / AVX512 可行性探索	58
1. 现状与基线	58
2. ECDSA 为什么适合（又为什么难）做 SIMD	58
3. 三条技术路线	59
路线 A: 标量算法升级（先决条件，无 SIMD）	59
路线 B: 批量验证 <code>ecdsa_p256_batch_verify</code> (MSM, 与指令集无关)	59
路线 C: SIMD 字段层（沿用 <code>fe_448_simd.hpp</code> 模式）	59
路线对比	60
4. 与现有代码的契合点	60
5. 风险与注意事项	61
6. 建议路线图	61
Phase 1 实测结果 (i7-13700K, MSVC Release, <code>benchmarks/bench_ecdsa_ossl.exe</code>)	61
Phase 1.5: P-256 特殊形式归约 (nistz256 风格, C++ 版)	62
Phase 1.6: MASM ADX 域核心 (已完成)	63
Phase 1.7: 三项收尾优化 (已完成)	63
Phase 2: ECDSA 接入 TLS 密钥交换 (已完成)	63
Phase 3: ECDHE 批量/多缓冲加速 (第二种方案, 本地可验证部分已完成)	64
Phase 3.1: 单次 ECDH 加速——点运算汇编无分支化 (已实现)	65
Phase 3.2: P-256 专用求逆加法链 + 汇编 (已实现)	66
Phase 3.3: 合并上游后的 flaky 堆损坏排查 (已解决)	66
7. 参考资料	66

JPSSL 维基

欢迎来到 **JPSSL** 维基。JPSSL 是一个使用 C++20 编写的高性能跨平台密码学库，同时支持 **CPU 优化**（AES-NI / AVX2 / AVX-512 / Montgomery 汇编）与可选的 **MUSA GPU 加速**（实验性）。

特性一览

- **对称加密**: AES-128/192/256 (ECB / CBC+PKCS7 / GCM / CCM)、ChaCha20-Poly1305、SM4 (ECB / CBC / GCM / CCM)
- **哈希 / MAC / KDF**: SHA-256、SHA-384/512、SHA3-256/384/512、SM3、HMAC (SHA-256/384/SM3)、HKDF (SHA-256/384/SM3)
- **非对称加密**: RSA-2048/4096 (PKCS#1 v1.5 / OAEP / PSS)、ECDSA P-256、Ed25519、Ed448、X25519、X448、SM2 (签名 / 验签 / 密钥交换)
- **证书**: X.509 v3 DER 编解码、自签名证书、证书链验证 (RFC 5280)，支持 RSA / Ed25519 / Ed448 / ECDSA / SM2 五种密钥类型
- **TLS**: TLS 1.2 (RFC 5246) 与 TLS 1.3 (RFC 8446) 完整握手、0-RTT / PSK 会话恢复、SNI 多证书管理、**RFC 8998 国密套件** (TLS_SM4_GCM_SM3 + SM2)
- **DTLS**: DTLS 1.2 (RFC 6347) 与 DTLS 1.3 (RFC 9147) 标准数据报 TLS，支持记录层 (epoch/seq + AEAD)、cookie、分片/重传/ACK、ECDHE (X25519 / P-256 / X448)、AES-128/256-GCM 与 ChaCha20-Poly1305；已与 OpenSSL 4 (DTLS 1.2) 和 wolfSSL 5.9.2 (DTLS 1.2/1.3) 双向互通验证
- **证书透明 (CT)**: 基于 RFC 6962 的国际 CT (SHA-256 + ECDSA P-256 / RSA) 与国密 CT (SM3 + SM2, 参考 GM/T 草案)
- **工具链**: [jpssl-cert](#) 证书命令行工具、[jpssl-crypt](#) 加解密命令行工具、TLS-over-TCP socket 封装层、HTTPS + CT 示例
- **性能**: 运行时 CPU 特性检测自动分派 (AES-NI / AVX2 / AVX-512 / SHA-NI / BMI2+ADX)，多数算法在基准中与 OpenSSL 持平或反超
- **平台支持**: Linux (x86_64 / ARM64)、Windows x64 (MSVC)、macOS arm64、iOS 13+ (arm64 XCFramework)、Android (arm64-v8a AAR)、HarmonyOS / OpenHarmony (arm64-v8a .so)

开始使用

- 新手引导 — 面向新手的完整入门路线：构建、第一个程序、常见问题
- 快速开始 — 环境要求、Linux / Windows 构建、安装与链接
- 构建选项与产物 — CMake 选项、静态/动态库、平台差异
- 命令行工具 — [jpssl-cert](#) 与 [jpssl-crypt](#) 用法

API 参考

- 算法支持总览
- 对称加密 API — AES、ChaCha20-Poly1305、SM4、SM4-GCM、SM4-CCM、Base64
- 哈希 / MAC / KDF API — SHA-2、SHA-3、SM3、HMAC、HKDF
- 非对称加密 API — RSA、ECDSA、Ed25519、X25519、Ed448、X448、SM2

- X.509 证书 API
- TLS API — TLS 1.2 / 1.3、0-RTT、RFC 8998 国密套件
- TLS socket 封装层
- 证书透明 (CT)

进阶主题

- MUSA GPU 加速
- 性能基准
- 测试与 CI

相关链接

- 项目主页: <https://github.com/DaChengTechnology/JPSSL>
- 更新日志: [CHANGELOG.md](#)

新手引导

本页是 JPSSL 的入门地图：从零开始，带你完成「获取代码 → 构建 → 运行第一个程序 → 找到需要的文档」全流程。如果你是第一次接触这个项目，建议按顺序阅读；如果只是查某个具体问题，可以直接跳到常见问题。

JPSSL 是什么？

JPSSL 是一个用 C++ 编写的高性能跨平台密码学库（要求 C++20），提供对称加密、哈希 / MAC / KDF、非对称加密、X.509 证书、TLS / DTLS 等能力，并内置国密（SM2 / SM3 / SM4）支持。它面向的是**开发者**：你通过 `#include` 头文件、链接库来使用它，而不是把它当作一个现成的加密工具。

快速了解它的定位与能力，见首页的「特性一览」与算法支持总览。

只想临时加密文件 / 生成证书？可以跳过 C++ 部分，直接使用附带的命令行工具 `jpssl-cert` 与 `jpssl-crypt`（见命令行工具）。

学习路线

阶段	目标	文档
1	了解功能与算法支持	首页、算法支持总览
2	准备环境、构建库	本页 + 快速开始
3	运行第一个程序	本页「第一个程序」
4	用命令行工具快速体验	命令行工具
5	在自己的项目里使用 API	对称加密 API、哈希 / MAC / KDF API、非对称加密 API、X.509 证书 API、TLS API、TLS socket 封装层
6	进阶：性能、测试、GPU	性能基准、测试与 CI、MUSA GPU 加速、ECDSA 性能与 SIMD 可行性

环境要求

依赖	版本要求	说明
C++ 编译器	支持 C++20: GCC 13+ / Clang 16+ / MSVC (VS 2019 16.10+ 或 VS 2022+)	库与公共头文件基于 C++20 (<code>std::span</code> 、 <code>constexpr</code> 等)
CMake	3.20+	构建系统
OpenSSL	可选	仅部分测试 / 基准的对比目标需要，库本身不依赖
MUSA SDK	4.3.0+ (可选)	实验性 GPU 加速，默认关闭，仅 Linux
平台	Linux / Windows / macOS / iOS / Android / HarmonyOS	各平台构建见平台构建

第 1 步：获取代码

```
git clone https://github.com/DaChengTechnology/JPSSL.git
cd JPSSL
```

第 2 步：构建

Linux / macOS

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . -j$(nproc)
```

Windows (MSVC)

在 **x64 Native Tools Command Prompt** 或 **Developer PowerShell** 中执行：

```
cmake -S . -B build-win -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-win
```

没有 Ninja 时可以直接生成 VS 工程：

```
cmake -S . -B build-win -G "Visual Studio 17 2022" -A x64
cmake --build build-win --config Release
```

构建产物与更多 CMake 选项见构建选项与产物。

第 3 步：运行测试（可选但推荐）

```
ctest --test-dir build --output-on-failure      # Linux / macOS
ctest --test-dir build-win --output-on-failure  # Windows
```

全部通过说明环境与构建没有问题。

第 4 步：运行你的第一个程序

把下面的代码保存为 **hello.cpp**。它演示两件最常见的事：计算 SHA-256 哈希，以及用 AES-256-GCM 加密并解密一段数据（GCM 是带认证的 AEAD 模式，适合绝大多数场景）。

```
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>

#include "aes.hpp"
#include "rand_os.hpp"
#include "sha256.hpp"

int main() {
    // 1) SHA-256 哈希
    const char* msg = "hello jpssl";
    uint8_t digest[32];
    jpssl::sha256(reinterpret_cast<const uint8_t*>(msg), std::strlen(msg), digest);
    std::printf("sha256(\"%s\") = %s\n", msg, jpssl::sha256_hex(digest).c_str());

    // 2) AES-256-GCM 认证加密 / 解密
    uint8_t key[32], iv[12], tag[16];
    if (!jpssl::os_rand_bytes(key, sizeof key) ||
        !jpssl::os_rand_bytes(iv, sizeof iv)) {
        std::fprintf(stderr, "os_rand_bytes failed\n");
        return 1;
    }

    jpssl::aes_context ctx;
    ctx.init(std::span<const uint8_t, 32>(key));

    const char* plaintext = "Hello, JPSSL!";
    std::vector<uint8_t> plain(plaintext, plaintext + std::strlen(plaintext));
```

```

std::vector<uint8_t> ct, recovered;

jpssl::aes_gcm_encrypt(ctx, iv, sizeof iv, plain, {}, ct, tag);
bool ok = jpssl::aes_gcm_decrypt(ctx, iv, sizeof iv, ct, {},
    tag, sizeof tag, recovered);

std::printf("aes-256-gcm decrypt: %s\n",
    ok && recovered == plain ? "OK" : "FAILED");
return ok && recovered == plain ? 0 : 1;
}

```

编译运行

Linux / macOS (动态链接) :

```

g++ -std=c++20 -O2 hello.cpp -linclude -Lbuild -ljpsl_cpu -o hello
LD_LIBRARY_PATH=build ./hello

```

Windows (x64 Native Tools / Developer PowerShell, 静态链接) :

```

cl /std:c++20 /utf-8 /EHsc /O2 /MD /linclude hello.cpp build-win\jpssl_cpu_static.lib bcrypt.lib
.\hello.exe

```

Windows 动态链接 (运行前把 **build-win** 加入 PATH, 或把 **jpssl_cpu.dll** 复制到 exe 旁边) :

```

cl /std:c++20 /utf-8 /EHsc /O2 /MD /linclude hello.cpp build-win\jpssl_cpu.lib
.\hello.exe

```

预期输出 (哈希值固定, 可用于确认结果正确) :

```

sha256("hello jpssl") = 59970df8f7466760058b3eb217af6db67abb9a7d95c30213ce150c8e84b0af3e
aes-256-gcm decrypt: OK

```

集成到 CMake 项目 (推荐)

用 **add_subdirectory** 或 **FetchContent** 引入后, 头文件目录与系统库 (Windows 上的 **bcrypt** / **ws2_32** / **crypt32**) 会自动传播, 不需要手动配置:

```

add_subdirectory(JPSSL)
target_link_libraries(your_app PRIVATE jpssl_cpu)      # 静态库
# target_link_libraries(your_app PRIVATE jpssl_cpu_shared) # 动态库

```

第 5 步: 用命令行工具快速体验

不想写代码时, 可以直接使用构建出的两个工具:

```

# 生成自签名证书 (ed25519)
jpssl-cert gen --cn example.com --key-type ed25519

# AES-256-GCM 加密文件 (密钥以十六进制传入)
jpssl-crypt encrypt --algo aes256gcm --key <hex-key> --in plain.txt --out cipher.bin

# 计算 SHA-256
jpssl-crypt hash --algo sha256 --in file.txt

```

完整用法见命令行工具。

常见问题

Q: 报错说找不到 `std::span` 或 `constexpr`?

A: 库要求 C++20。用 CMake 构建时项目已自动设置; 手动编译请加 **-std=c++20** (GCC / Clang) 或 **/std:c++20** (MSVC) 。

Q: MSVC 编译时出现 C4819 警告 (中文注释乱码) ?

A: 源文件按 UTF-8 处理即可, 手动用 `cl` 编译时加 `/utf-8`。通过 CMake 构建时项目已全局添加该选项。

Q: Windows 静态链接报 LNK2038 RuntimeLibrary 不匹配?

A: 库是用 `/MD` (动态 CRT) 构建的, 手动编译时请加 `/MD` 保持一致。

Q: 链接报找不到 BCryptGenRandom 等符号?

A: Windows 需要链接 `bcrypt.lib` (TLS socket 还需要 `ws2_32.lib`, X.509 还需要 `crypt32.lib`)。用 CMake target 引入时这些会自动处理。

Q: 密钥 / IV 应该从哪里来?

A: 用 `jpssl::os_rand_bytes()` (Windows 走 `BCryptGenRandom`, Linux 走 `/dev/urandom`)。不要用 `std::random_device` 生成密钥——MSVC 上的实现是确定性的。

Q: 需要自己写 CPUID 检测来启用硬件加速吗?

A: 不需要。库在运行时自动按 CPU 特性分派 (AES-NI / AVX2 / AVX-512 / SHA-NI 等), 不支持的 CPU 会自动回退, 见构建选项与产物。

Q: MUSA GPU 加速怎么用?

A: 实验性功能, 默认关闭、仅 Linux、需要 MUSA SDK 4.3.0+, 构建时加 `-DJP_ENABLE_MUSA=ON`。详见 MUSA GPU 加速。

Q: 链接时用哪个库?

A: `jpssl_cpu` (静态) 或 `jpssl_cpu_shared` (动态)。命令行工具 `jpssl-cert` / `jpssl-crypt` 在 `make install` 后位于 `bin/`。

Q: 使用密码学时要注意什么?

A: 几条基本建议: 优先使用 AEAD 模式 (如 AES-GCM / ChaCha20-Poly1305 / SM4-GCM); **永远不要**用同一个 `nonce` / `IV` 加密两条不同的消息; 不要用无认证的 ECB 模式加密多块数据; 密钥与 IV 用随机源生成。库的接口已经尽量降低误用风险, 但正确使用仍是调用方的责任。

下一步

- 想了解所有 API 与示例: 从对称加密 API 和哈希 / MAC / KDF API 开始, 再到非对称加密 API
- 想跑 HTTPS / TLS: 先看 TLS API 与 TLS socket 封装层, 以及 `examples/` 目录下的完整示例
- 想了解性能与正确性保证: 性能基准、测试与 CI
- 想了解国密支持: 算法支持总览与 RFC 8998 国密套件

快速开始

环境要求

依赖	版本要求	说明
C++ 编译器	GCC 13+ / Clang 16+ / MSVC (VS 2019 16.10+ 或 VS 2022+)	C++20 标准, CMAKE_CXX_STANDARD=20
CMake	3.20+	构建系统
OpenSSL	可选	仅部分测试 / 基准的对比目标需要, 库本身不依赖
MUSA SDK	4.3.0+ (可选)	实验性 GPU 加速, 默认关闭, 仅 Linux
硬件	x86_64 (桌面)	AES-NI / AVX2 / AVX-512 / SHA-NI / BMI2+ADX 可选; ARM64 走 NEON/crypto 路径
平台	Linux / Windows / macOS / iOS / Android / HarmonyOS	各平台构建见 平台构建

Linux / macOS 构建

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
```

启用 MUSA GPU 加速 (实验性, 仅 Linux, 需要 MUSA SDK 4.3.0+) :

```
cmake .. -DCMAKE_BUILD_TYPE=Release -DJ_P_ENABLE_MUSA=ON
```

运行主测试程序与全部单元测试:

```
# 运行主测试程序
LD_LIBRARY_PATH=/usr/local/musa/lib ./jpssl-test

# 运行全部单元测试 (CTest, 目标定义在 tests/CMakeLists.txt)
LD_LIBRARY_PATH=/usr/local/musa/lib ctest --output-on-failure
```

安装到系统 (可选, 安装命令行工具 **jpssl-cert** / **jpssl-crypt** 到 **bin/**) :

```
sudo make install
```

Windows 构建 (MSVC)

支持 Windows x64 + MSVC (VS 2019 16.10+ / VS 2022+ / Build Tools) , 通过 CMake + Ninja 或 VS 解决方案构建。请在 **"x64 Native Tools Command Prompt"** 或 **Developer PowerShell** 中执行:

```
# Ninja 生成器
cmake -S . -B build-win -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-win

# 或直接生成 VS 工程
# cmake -S . -B build-win -G "Visual Studio 17 2022" -A x64

# 运行测试
ctest --test-dir build-win --output-on-failure
```

链接到你的项目

构建默认同时生成静态库和动态库：

库文件	类型	说明
libjpssl_cpu.a	静态库	CPU 密码学库 (Linux)
libjpssl_cpu.so	动态库	CPU 密码学库 (Linux 共享)
libjpssl_musa.a / libjpssl_musa.so	GPU 库	MUSA GPU 加速库 (仅启用 JP_ENABLE_MUSA)
jpssl_cpu_static.lib	静态库	Windows 静态库 (避免与导入库同名冲突)
jpssl_cpu.dll / jpssl_cpu.lib	动态库	Windows 共享库及导入库

命令行链接动态库：

```
g++ -std=c++20 your_app.cpp -I./include -L./build -ljpssl_cpu -o your_app
LD_LIBRARY_PATH=./build ./your_app
```

使用 CMake 的 `FetchContent` 或 `add_subdirectory` 引入时，链接目标为 `jpssl_cpu` (静态) 或 `jpssl_cpu_shared` (动态)，头文件目录 `include/` 已通过 `target_include_directories(... PUBLIC ...)` 传播。

下一步

- 查看 构建选项与产物 了解所有 CMake 开关
- 查看 算法支持总览 了解支持的算法
- 查看 命令行工具 快速体验 `jpssl-cert` / `jpssl-crypt`

构建选项与产物

CMake 选项

选项	默认	说明
JP_ENABLE_MUSA	OFF	MUSA GPU 加速 (实验性, 仅 Linux, 需要 MUSA SDK 4.3.0+)
JP_ENABLE_BENCH	OFF	构建 benchmarks/ 基准程序
JP_ENABLE_OPENMP	ON	OpenMP 并行 (CPU 批量 RSA 模幂, RSA 解密双线程)
JP_ENABLE_AVX2	ON	AVX2 GCM (4 路并行) + SHA-512 SIMD 消息调度 + ChaCha20/Poly1305/X448/Ed448/Ed25519 SIMD 路径
JP_ENABLE_AVX512	ON	AVX512 VAES GCM (8 路并行) + Base64 AVX-512 + RSA 批量模幂

```
# 启用 MUSA GPU 加速 (需要 MUSA SDK 4.3.0+)
cmake -B build -DJP_ENABLE_MUSA=ON

# 禁用所有 SIMD 加速 (纯标量回退)
cmake -B build -DJP_ENABLE_AVX2=OFF -DJP_ENABLE_AVX512=OFF

# 构建基准程序
cmake -B build -DJP_ENABLE_BENCH=ON
```

构建产物

构建默认同时生成静态库与动态库:

库文件	类型	说明
libjpssl_cpu.a	静态库	CPU 密码学库
libjpssl_cpu.so	动态库	CPU 密码学库 (SOVERSION 1)
libjpssl_musa.a / libjpssl_musa.so	静态/动态	GPU 加速库 (仅 JP_ENABLE_MUSA=ON)
jpssl_cpu_static.lib	静态库	Windows 静态库
jpssl_cpu.dll + jpssl_cpu.lib	动态库	Windows 共享库与导入库

命令行工具 (**make install** 后安装到 **bin/**) :

工具	说明
jpssl-cert	X.509 v3 证书生成 / 查看 / 验证
jpssl-crypt	加密 / 解密 / 哈希 / HMAC / 随机数
jpssl-test	主测试程序

平台与编译器注意事项

Windows (MSVC)

- **UTF-8 源文件**: 全局编译选项包含 `/utf-8`, 避免中文注释引发的 C4819 警告。
- **128 位整数兼容层**: MSVC 没有 GCC 的 `__uint128_t`, 通过 `/FI` 强制包含 `include/jpssl_platform.hpp`, 以 `jp_uint128` (基于 `_umul128` / `_addcarry_u64` / `_udiv128`, 全部内联) 提供等价语义。业务代码无需改动。
- **随机数**: 统一走 `include/rand_os.hpp` (`jpssl::os_rand_bytes`) ——Windows 用 `BCryptGenRandom`, Linux 用 `/dev/urandom`。MSVC 的 `std::random_device` 是确定性的, 不用于密钥 / 签名 nonce。
- **CPU 特性检测**: `include/cpu_features.hpp` 在 MSVC 下使用 `__cpuidex` / `_xgetbv` 做运行时检测 (AES-NI / AVX2 / AVX512 / SHA-NI), 硬件加速分派在 Windows 上同样生效。
- **RSA Montgomery 汇编**: CIOS Montgomery 乘法走手写 MASM (`src/rsa_mont_asm_win.asm`, MULX 加速); CRT 私钥解密使用半尺寸汇编路径 (`mont_mul_half_asm`)。
- **SM3 汇编**: MSVC x64 下 `sm3_win.asm` (MASM) 提供 64 轮全展开压缩函数, `sm3_cf` 自动走汇编。
- **动态库导出**: 通过 `WINDOWS_EXPORT_ALL_SYMBOLS` 导出全部公共符号, 无需手写 `__declspec(dllexport)`。
- **链接系统库**: Windows 自动链接 `bcrypt` (随机数) 与 `ws2_32` (TLS socket)。
- **MUSA**: 不支持 Windows, `-DJP_ENABLE_MUSA=ON` 会被自动禁用并提示。

Linux (GCC / Clang)

- x86_64 下全局编译标志包含 `-maes -mavx2 -madx`。
- RSA Montgomery 汇编走 GCC 内联汇编 (`src/rsa_mont_asm.cpp`); 运行时 CPU 不支持 BMI2/ADX 时自动回退到标量 `_umul128` 实现, CPUID 结果进程内缓存。
- MUSA 模块通过 `/usr/local/musa/cmake` 的 `find_package(MUSA)` 集成, `mcc` 编译 `--offload-arch=mp_21 / mp_22 / mp_31`。

条件编译与运行时调度

大部分 SIMD / 硬件加速路径都是**运行时按 CPUID 自动分派**, 而非编译期硬编码:

- AES-GCM: 软件 / AVX2 (4 路) / AVX512 VAES (8 路) 自动分派 (`aes_gcm_auto.cpp`)
- Base64: 标量 / AVX2 / AVX-512 (含 BW) 自动分派
- SHA-1 批量哈希: 标量 / AVX2 (8 路) / AVX-512 (16 路) 自动分派 (`sha1_batch`)
- SM4-GCM: GHASH 按 PCLMULQDQ 支持自动分派 (`sm4_gcm_dispatch.cpp`)
- RSA 批量模幂: 标量 / AVX2 / AVX-512 (`rsa_batch_dispatch.cpp`)
- Ed25519 / Ed448 批量验签: CPU / AVX2 / AVX-512 (`ed25519_batch_dispatch.cpp` / `ed448_batch_dispatch.cpp`)
- SHA-512: CPU / SSE4.1 SIMD 消息调度

即使编译时启用了 AVX2/AVX-512, 在不支持的 CPU 上也会自动回退到可用路径, 不需要运行时检测代码。

依赖

- **C++20** (GCC 13+ / Clang 16+ / MSVC)
- **CMake** 3.20+
- **OpenSSL** (可选, 仅 OpenSSL 对比测试 / 基准需要; 未安装时自动跳过)
- **MUSA SDK** 4.3.0+ (可选, 实验性 GPU 加速, 默认关闭, 仅 Linux)
- **x86_64** (AES-NI / AVX2 / AVX512, 可选)

移动端 / 桌面端平台构建

iOS (XCFramework) 、 macOS (dylib/.so/framework) 、 Android (AAR) 与 HarmonyOS / OpenHarmony (.a/.so) 的一键构建脚本与产物说明见平台构建 (iOS / macOS / Android / HarmonyOS) 。

平台构建 (iOS / macOS / Android / HarmonyOS)

除 Linux / Windows 的常规 CMake 构建外，仓库为移动端与桌面端提供一键构建脚本与固定产物，且每个平台都有 GitHub Actions 工作流守护（见 测试与 CI）。

iOS (XCFramework, arm64)

前置：macOS + Xcode 15+（C++20 协程需要 Apple Clang 15+）、CMake 3.20+、Ninja。

```
./ios/build-xcframework.sh      # 全部（真机 + 模拟器）
./ios/build-xcframework.sh device  # 仅真机
./ios/build-xcframework.sh simulator # 仅模拟器
```

产物：**ios/JPSsl.xcframework**（**JPSslC.framework** 静态片，含 **Headers/jpssl.h**、**Modules/module.modulemap** 与 **Info.plist**）。Swift 侧可 **import JPSslC** 使用 **extern-C** 桥接 API，或通过 **ios/Package.swift** 引入封装层 **JPSsl**。
最低部署版本 iOS 13.0 / arm64。

macOS (arm64: 动态库 + framework)

前置：macOS + Xcode Command Line Tools、CMake、Ninja（Apple Silicon 或 x64 均可，**-DCMAKE_OSX_ARCHITECTURES=arm64** 固定目标切片）。

```
./macos/build.sh
```

产物：

- **build-macos/libjpssl_cpu.dylib** —— 标准 macOS 动态库
- **build-macos/libjpssl_cpu.so** —— Mach-O dylib 的同名副本，便于 FFI / **dlopen** 按 **.so** 约定加载
- **build-macos/libjpssl_cpu.a** —— 静态库
- **macos/JPSslC.framework** —— 动态 framework（**@rpath** install name、**Headers/Modules/Info.plist**、ad-hoc 签名）

链接示例：

```
clang app.c -lios/bridge -Fmacos -framework JPSslC -o app
# 或直接链接动态库
clang app.c -lios/bridge -Lbuild-macos -ljpssl_cpu -o app
```

Android (AAR, arm64-v8a)

android/ 为 Gradle 工程（AGP 8.2.2 / Kotlin 1.9.22 / NDK r26b，仅 arm64-v8a，C++20）。构建需要 JDK 17 + Android SDK（platform 34 / build-tools 34 / NDK 26.1.10909125）。

```
gradle -p android assembleRelease
```

产物：**android/jpssl/build/outputs/aar/jpssl-release.aar**，内含 **jni/arm64-v8a/libjpssl.so** 与 **libc++_shared.so**。
Java / Kotlin API 见 **io.github.jpssl**（**Jpssl.java** / **Jpssl.kt**）。

HarmonyOS / OpenHarmony (arm64-v8a .a / .so)

前置：OpenHarmony SDK 的 **native** 目录（含 **build/cmake/ohos.toolchain.cmake**，例如 DevEco Studio SDK）。

```
export OHOS_NDK_HOME=<sdk>/.../native
cmake -S . -B build-ohos -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/ohos.cmake \
  -DOHOS_ARCH=arm64-v8a -DCMAKE_BUILD_TYPE=Release
cmake --build build-ohos
```

产物：**build-ohos/libjpssl_cpu.a** 与 **libjpssl_cpu.so**（OpenHarmony NDK 不提供可执行文件链接，测试 / CLI 自动跳过）。支持

OHOS_ARCH=arm64-v8a | armeabi-v7a | x86_64、**OHOS_STL=c++_shared | c++_static**、**OHOS_API**（默认 12）。

命令行工具

构建后生成两个命令行工具，`make install` 后安装到 `bin/` 目录：

工具	说明	源文件
<code>jpssl-cert</code>	X.509 v3 证书生成 / 查看 / 验证	<code>src/cmd/jpssl_cert.cpp</code>
<code>jpssl-crypt</code>	加密 / 解密 / 哈希 / HMAC / Base64 / 随机数	<code>src/cmd/jpssl_crypt.cpp</code>

jpssl-cert — 证书工具

```
# 生成自签名 X.509 v3 证书 + 私钥 (有效期默认 365 天)
# 默认输出到 ~/.ssh/cert.der 与 ~/.ssh/key.bin (目录自动创建, 私钥权限 0600)
jpssl-cert gen --cn example.com --key-type ed25519
# 支持的密钥类型: ed25519 | ecdsa | sm2 | rsa2048 | ed448
# 用 --days 指定有效期 (gen / tlsgen 均支持, 默认 365 天)
# 用 --out / --key-out 指定其他位置 (支持 ~ 展开)
jpssl-cert gen --cn example.com --key-type ed25519 --days 90 --out ~/certs/cert.der --key-out
~/certs/key.bin
# 私钥默认输出为 PKCS#8 PEM (-----BEGIN PRIVATE KEY-----, 与 openssl genpkey 一致;
# Ed25519/Ed448 为 RFC 8410, ECDSA/SM2 内嵌 SEC1, RSA 内嵌完整 PKCS#1) ;
# 需要二进制时加 --key-format der

# 查看证书信息 (支持 DER 或 PEM)
jpssl-cert info --cert cert.der
jpssl-cert info --cert cert.pem

# 查看私钥 (PKCS#8 / PKCS#1 / SEC1 / [RFC 8410](https://www.rfc-editor.org/rfc/rfc8410) PEM)
jpssl-cert key --key key.pem
# 查看加密私钥 (PBES2, 需 --pass 密码)
jpssl-cert key --key encrypted.pem --pass your-password

# 查看 CSR (PKCS#10 PEM)
jpssl-cert csr --csr request.csr

# 验证证书链 (leaf → root) (支持 DER 或 PEM)
jpssl-cert verify --cert leaf.der --ca root.der
# 多级链: --ca 可多次指定中间 CA
jpssl-cert verify --cert leaf.der --ca intermediate.der --ca root.der

# 通过 TLS API 生成证书 (等价于 tls_make_x509_self_signed)
jpssl-cert tlsgen --cn example.com --key-type ecdsa
# 指定有效期 10 年
jpssl-cert tlsgen --cn example.com --key-type ecdsa --days 3650 --out cert.der --key-out key.bin
```

jpssl-crypt — 加密 / 哈希工具

```
# AES-256-GCM 加密 ([FIPS 197](https://csrc.nist.gov/publications/detail/fips/197/final) + [NIST SP
800-38D](https://csrc.nist.gov/pubs/sp/800/38d/final), 输出格式: IV || 密文 || Tag)
jpssl-crypt encrypt --algo aes256gcm --key <hex-key> --in plain.txt --out cipher.bin

# ChaCha20-Poly1305 加密 ([RFC 8439](https://www.rfc-editor.org/rfc/rfc8439))
jpssl-crypt encrypt --algo chacha20 --key <hex-key> --in plain.txt --out cipher.bin

# 解密 (自动从文件提取 IV 和 Tag)
jpssl-crypt decrypt --algo aes256gcm --key <hex-key> --in cipher.bin --out plain.txt

# 哈希
jpssl-crypt hash --algo sha1 --in file.txt # SHA-1 (20 bytes)
jpssl-crypt hash --algo sha256 --in file.txt
jpssl-crypt hash --algo sm3 --in file.txt # 国密 SM3

# HMAC ([RFC 2104](https://www.rfc-editor.org/rfc/rfc2104) / [FIPS
198-1](https://csrc.nist.gov/pubs/fips/198-1/final))
jpssl-crypt hmac --algo sha256 --key <hex-key> --in file.txt

# Base64 编码 / 解码 ([RFC 4648](https://www.rfc-editor.org/rfc/rfc4648))
jpssl-crypt b64encode --in file.bin --out file.b64
jpssl-crypt b64decode --in file.b64 --out file.bin

# 生成随机字节 (十六进制输出)
jpssl-crypt rand 32
```

支持的算法:

- **加密**: **aes256gcm** (AES-256-GCM AEAD) 、 **chacha20** (ChaCha20-Poly1305 AEAD)
- **哈希**: **sha1**、**sha256**、**sha512**、**sha3-256**、**sha3-512**、**sm3**
- **HMAC**: **sha256**、**sha384**、**sm3**
- **Base64**: **b64encode** (二进制 → [RFC 4648](#) 文本) 、 **b64decode** (文本 → 二进制)

密钥、IV、Tag 均以十六进制字符串传入, AAD 认证数据可用 **--aad <hex>** 指定。

b64decode 容忍文本文件中的空白字符 (行尾换行、空格、tab) , 解码失败时返回非零退出码; 未指定 **--out** 时输出到 stdout (编码输出带换行, 解码输出原始二进制) 。

算法支持总览

算法	模式	CPU 加速	GPU 加速
AES-128/256	ECB、CBC+PKCS7、GCM、CCM	AES-NI (约 5 GB/s)、AVX2/AVX512 GCM	ECB kernel (实验性)
GHASH / GF(2 ¹²⁸)	GCM 认证原语 (一次性/增量/GCM 完整哈希)	PCLMULQDQ	—
ChaCha20-Poly1305	流加密、AEAD	AVX2 / AVX-512 多块并行	Keystream kernel (实验性)
RSA 2048/4096	PKCS#1 v1.5、OAEP、PSS	Montgomery CIOS + MULX 汇编、批量 AVX2/AVX512	批量模幂 (实验性)
SHA-1	哈希 (FIPS 180-4)	AVX2 8 路 / AVX-512 16 路多缓冲批量哈希	—
SHA-256	哈希	SHA-NI	—
SHA-384/512	哈希 (FIPS 180-4)	SSE4.1 消息调度	GPU kernel (实验性)
SHA3-256/384/512	哈希 (FIPS 202, Keccak)	—	—
HMAC-SHA256/SHA384/SM3	MAC	—	—
HKDF-SHA256/SHA384/SM3	TLS 1.3 密钥派生	—	—
X25519	ECDH 密钥交换	AVX-512	—
X448	ECDH 密钥交换	AVX2 / AVX-512	—
Ed25519	数字签名 (EdDSA)	AVX-512、批量验签	—
Ed448	数字签名 (EdDSA)	AVX2 / AVX-512、批量验签	—
ECDSA P-256	数字签名 (secp256r1)	—	—
SM2	数字签名 / 密钥交换 (sm2p256v1, GM/T 0003)	Montgomery(CIOS) + wNAF-5 标量乘 + Shamir 双标量	—
SM3	密码杂凑 (256-bit, GM/T 0004)	MSVC x64 MASM 标量汇编	—
SM4	分组密码 (128-bit, GM/T 0002)，ECB / CBC / GCM / CCM	T 表加速标量核心，SM4-GCM 使用 PCLMULQDQ 快速 GHASH	—
X.509 v3	证书 DER / PEM 编解码 (RFC 5280)、私钥读取 (PKCS#8 / PKCS#1 / SEC1 / 加密 PBES2)、CSR (PKCS#10)、自签名、证书链、SAN / KeyUsage / BasicConstraints	—	—
TLS 1.2/1.3	完整握手、密码套件协商、ECDHE/RSA、0-RTT、 RFC 8998 国密套件	AVX2/AVX512 GCM	—

算法	模式	CPU 加速	GPU 加速
Base64	RFC 4648 编解码	AVX2 (约 21-22 GB/s 编码)、AVX-512 (约 17 GB/s 解码)	—
证书透明 (CT)	RFC 6962 国际 (SHA-256 + ECDSA P-256 / RSA) + 国密 (SM3 + SM2)	—	—

TLS 密码套件

TLS 1.3

- AES-128-GCM / AES-256-GCM (SHA-256 / SHA-384)
- ChaCha20-Poly1305 (SHA-256)
- AES-128-CCM
- [RFC 8998](#) 国密: [TLS_SM4_GCM_SM3](#) + SM2 签名
- 证书签名算法: Ed25519、Ed448、ECDSA P-256、RSA-2048/4096、SM2
- 密钥交换: X25519、X448 (基于前向保密 ECDHE)
- 会话恢复: PSK / NewSessionTicket / 0-RTT early data

TLS 1.2 ([RFC 5246](#))

套件	密钥交换	加密	哈希
0xC02C	ECDHE-ECDSA	AES-256-GCM	SHA-384
0xC030	ECDHE-RSA	AES-256-GCM	SHA-384
0xC02B	ECDHE-ECDSA	AES-128-GCM	SHA-256
0xC02F	ECDHE-RSA	AES-128-GCM	SHA-256
0xCCA9	ECDHE-ECDSA	ChaCha20-Poly1305	SHA-256
0xCCA8	ECDHE-RSA	ChaCha20-Poly1305	SHA-256
0x009C	RSA	AES-128-GCM	SHA-256
0x009D	RSA	AES-256-GCM	SHA-384

标准合规

- AES: [FIPS 197](#); GCM: [NIST SP 800-38D](#); CCM: [NIST SP 800-38C](#); ChaCha20-Poly1305: [RFC 8439](#)
- SHA-1 / SHA-2: [FIPS 180-4](#); SHA-3: [FIPS 202](#); HMAC: [RFC 2104](#) / [FIPS 198-1](#); HKDF: [RFC 5869](#)
- RSA: PKCS#1 v1.5 / OAEP / PSS ([RFC 8017](#))
- Ed25519 / Ed448: RFC 8032 ([Ed25519](#)) ; X25519 / X448: [RFC 7748](#)
- ECDSA: secp256r1 ([FIPS 186-4](#))
- X.509: [RFC 5280](#)

- TLS 1.2: [RFC 5246](#); TLS 1.3: [RFC 8446](#); 国密 TLS: [RFC 8998](#)
- CT: [RFC 6962](#) (国际) / [RFC 9162](#) 一致性证明 (验证按 §2.1.4.2) / GM/T 草案 (国密)
- 国密: GM/T 0002-2012 (SM4) 、GM/T 0003-2012 (SM2) 、GM/T 0004-2012 (SM3) , 详见 [RFC 8998 国密套件](#) 与 [GM/T 标准说明](#)

对称加密 API

所有头文件位于 `include/` 目录，命名空间为 `jpssl`（部分 API 为全局函数，见各头文件）。

AES ([FIPS 197](#))

头文件: `include/aes.hpp`

```
#include "aes.hpp"
```

```
aes_context ctx;  
ctx.init(std::span<const uint8_t,16>(key));
```

```
aes_cbc_encrypt(ctx, iv, plaintext, ciphertext);  
aes_gcm_encrypt(ctx, iv, 12, plaintext, aad, ct, tag);
```

支持的密钥长度: 128 / 192 / 256 位。支持模式:

- **ECB**: 单块 / 批量加密
- **CBC**: PKCS#7 填充 (`aes_cbc_encrypt` / `aes_cbc_decrypt`)
- **GCM**: AEAD 认证加密 ([NIST SP 800-38D](#), 软件 / AVX2 / AVX512 VAES 自动分派, `aes_gcm_auto.cpp`)
- **CCM**: AEAD 认证加密 ([NIST SP 800-38C](#), `include/aes_ccm.hpp`)

硬件加速: AES-NI + PCLMULQDQ (GCM GHASH) ; AVX512 VAES GCM 8 路并行。

GHASH / GF(2¹²⁸) (GCM 认证核心, 见 [NIST SP 800-38D](#), 可单独使用)

```
uint8_t H[16], S[16];          // H = AES_encrypt(K, 0128)
```

```
// 一次性 GHASH (末尾块自动补零)  
ghash(H, data, S);
```

```
// 增量 GHASH (流式, 分块喂入)  
ghash_ctx g;  
ghash_init(&g, H);  
ghash_update(&g, aad.data(), aad.size());  
ghash_update(&g, ciphertext.data(), ciphertext.size());  
ghash_final(&g, S);
```

```
// 完整 GCM 认证哈希 (含 AAD/密文分段补零与长度块, 输出 GCM 的 S 值)  
gcm_ghash(H, aad, ciphertext, S);  
// 完整 GCM 标签: tag = S ^ AES_encrypt(K, J0), J0 = IV || 031 || 1
```

底层原语 `gf128_mul` (PCLMULQDQ 加速的 GF(2¹²⁸) 乘法, NIST 大端序约定) 同样公开, 可用于自定义认证构造。

ChaCha20-Poly1305 ([RFC 8439](#))

头文件: `include/chacha20_poly1305.hpp`

```
#include "chacha20_poly1305.hpp"
```

```
// AEAD 加密 ([RFC 8439](https://www.rfc-editor.org/rfc/rfc8439))  
chacha20_poly1305_encrypt(key, nonce, plaintext, aad, ct, tag);
```

```
// 纯流加密 (ChaCha20 keystream XOR)  
chacha20_stream_xor(key, nonce, input, output);
```

硬件加速：AVX2 / AVX-512 多块并行 ([chacha20_avx2.cpp](#) / [chacha20_avx512.cpp](#)) , Poly1305 有 AVX2 路径 ([poly1305_avx2.cpp](#)) 。

SM4 ([GM/T 0002-2012](#))

头文件：[include/sm4.hpp](#)、[include/sm4_gcm.hpp](#)、[include/sm4_ccm.hpp](#)

```
#include "sm4.hpp"

uint8_t key[16];
sm4_ctx ctx; sm4_init(&ctx, key);

// 单块
uint8_t cipher[16], recovered[16];
sm4_encrypt_block(&ctx, plain, cipher);
sm4_decrypt_block(&ctx, cipher, recovered);

// ECB
std::vector<uint8_t> ct(plain.size());
sm4_ecb_encrypt(&ctx, std::span<const uint8_t>(plain), std::span<uint8_t>(ct));

// CBC + PKCS#7
uint8_t iv[16] = {};
auto ct = sm4_cbc_encrypt(&ctx, iv, std::span<const uint8_t>(msg, len));
auto pt = sm4_cbc_decrypt(&ctx, iv, std::span<const uint8_t>(ct));
```

SM4-GCM AEAD ([NIST SP 800-38D](#) 框架, SM4 见 [GM/T 0002](#))

```
#include "sm4_gcm.hpp"

uint8_t key[16], iv[12], tag[16];
sm4_ctx ctx; sm4_init(&ctx, key);

std::vector<uint8_t> ct;
sm4_gcm_encrypt(&ctx, iv, 12,
    std::span<const uint8_t>(plaintext, pt_len),
    std::span<const uint8_t>(aad, aad_len),
    ct, tag, 16);

std::vector<uint8_t> recovered;
bool ok = sm4_gcm_decrypt(&ctx, iv, 12,
    std::span<const uint8_t>(ct),
    std::span<const uint8_t>(aad, aad_len),
    tag, 16, recovered);
```

SM4-GCM 遵循 [NIST SP 800-38D](#), GHASH 使用 PCLMULQDQ 快速路径 ([sm4_gcm_dispatch.cpp](#) 自动分派) , 修复了非 96 位 IV 的 J0 构造 (与 NIST SP 800-38D 一致) 。

SM4-CCM AEAD ([NIST SP 800-38C](#) 框架, SM4 见 [GM/T 0002](#))

```
#include "sm4_ccm.hpp"

uint8_t key[16], nonce[12], tag[16];
sm4_ctx ctx; sm4_init(&ctx, key);

std::vector<uint8_t> ct;
sm4_ccm_encrypt(&ctx, nonce, 12,
    std::span<const uint8_t>(plaintext, pt_len),
    std::span<const uint8_t>(aad, aad_len),
    ct, tag, 16);

std::vector<uint8_t> recovered;
bool ok = sm4_ccm_decrypt(&ctx, nonce, 12,
    std::span<const uint8_t>(ct),
    std::span<const uint8_t>(aad, aad_len),
    tag, 16, recovered);
```

Base64 ([RFC 4648](#))

头文件: [include/base64.hpp](#)

```
#include "base64.hpp"

std::string b64 = base64_encode(data, len);
auto decoded = base64_decode(b64); // std::optional<std::vector<uint8_t>>
```

运行时按 AVX-512 (含 BW) > AVX2 > 标量自动分派。AVX2 实测编码约 21–22 GB/s、解码约 17 GB/s (i7-13700K, 对比标量编码约 3 GB/s、解码约 0.2 GB/s)。尾部与 = 填充仍走标量, 语义与常规 base64 完全一致。

哈希 / MAC / KDF API

SHA-1 ([FIPS 180-4](#))

头文件: [include/sha1.hpp](#)

```
#include "sha1.hpp"

sha1_ctx ctx; sha1_init(&ctx);
sha1_update(&ctx, data, len);
uint8_t digest[20]; sha1_final(&ctx, digest);
std::string hex = sha1_hex(digest); // "a9993e364706816aba3e25717850c26c9cd0d89d"
```

加速: [src/sha1_avx2.cpp](#) (8 路多缓冲, YMM 每 lane 一条消息) 与 [src/sha1_avx512.cpp](#) (16 路多缓冲, ZMM)。等长批量哈希可通过 [sha1_batch](#) 自动分派:

```
const uint8_t* msgs[16] = {...};
uint8_t outs[16][20];
sha1_batch(msgs, len, &outs[0][0], 16); // AVX-512 16 路 > AVX2 8 路 > 标量
```

SHA-256 ([FIPS 180-4](#))

头文件: [include/sha256.hpp](#)

```
#include "sha256.hpp"

sha256_ctx ctx; sha256_init(&ctx);
sha256_update(&ctx, data, len);
uint8_t digest[32]; sha256_final(&ctx, digest);
```

硬件加速: SHA-NI ([sha256_sha_ni.cpp](#), `__x86_64__` || `_M_X64` 时编译)。

SHA-384 / SHA-512 ([FIPS 180-4](#))

头文件: [include/sha512.hpp](#)

```
#include "sha512.hpp"

sha512_ctx ctx;
sha512_init(&ctx); // 或 sha384_init 得到 SHA-384
sha512_update(&ctx, data, len);

uint8_t hash[64]; sha512_final(&ctx, hash); // 输出: SHA-512 64 字节; SHA-384 48 字节
std::string hex = sha512_hex(hash); // "e718483d0ce769..."
```

硬件加速: SSE4.1 SIMD 消息调度 ([sha512_opt.cpp](#))。

SHA3-256 / 384 / 512 ([FIPS 202](#))

头文件: [include/sha3.hpp](#)

```
#include "sha3.hpp"

sha3_ctx ctx;
sha3_256_init(&ctx); // 或 sha3_384_init / sha3_512_init
sha3_update(&ctx, data, len);

uint8_t hash[64]; sha3_final(&ctx, hash);
```

SM3 ([GM/T 0004-2012](#))

头文件: [include/sm3.hpp](#)

```
#include "sm3.hpp"

// 一次性哈希
uint8_t digest[32]; sm3_hash(digest, (const uint8_t*)data, len);

// 增量哈希
sm3_ctx ctx; sm3_init(&ctx);
sm3_update(&ctx, data, len);
sm3_final(&ctx, digest);
std::string hex = sm3_hex(digest);
```

性能: MSVC x64 下 **sm3_cf** 自动走 MASM 标量汇编 ([src/sm3_win.asm](#), 64 轮全展开), 8 KiB 吞吐约 390 MB/s; GCC/Clang 下使用 8 轮展开 + 常量折叠, 吞吐与 OpenSSL 持平。

HMAC ([RFC 2104](#) / [FIPS 198-1](#))

头文件: [include/hmac.hpp](#)

```
#include "hmac.hpp"

uint8_t mac[32]; hmac_sha256(key, 32, msg, len, mac);
uint8_t mac384[48]; hmac_sha384(key, 48, msg, len, mac384);
uint8_t mac_sm3[32]; hmac_sm3(key, 32, msg, len, mac_sm3);
```

HKDF ([RFC 5869](#))

头文件: [include/hkdf.hpp](#)

```
#include "hkdf.hpp"

// SHA-256
uint8_t prk[32]; hkdf_extract(salt, 16, ikm, 32, prk);
uint8_t okm[64]; hkdf_expand(prk, info, 8, okm, 64);

// SHA-384 (用于 TLS 1.3 AES-256-GCM 等套件)
uint8_t prk384[48]; hkdf_extract_sha384(salt, 16, ikm, 32, prk384);
uint8_t okm384[64]; hkdf_expand_sha384(prk384, info, 8, okm384, 64);

// SM3 (用于 TLS 1.3 [RFC 8998](https://www.rfc-editor.org/rfc/rfc8998) 国密套件)
uint8_t prk_sm3[32]; hkdf_extract_sm3(salt, 16, ikm, 32, prk_sm3);
uint8_t okm_sm3[64]; hkdf_expand_sm3(prk_sm3, info, 8, okm_sm3, 64);
```

随机数

头文件: [include/rand_os.hpp](#)

```
#include "rand_os.hpp"

// Windows: BCryptGenRandom; Linux: /dev/urandom
jpsl::os_rand_bytes(buffer, len);
```

所有密钥生成 / 签名 nonce 均使用该操作系统随机源, **不要**使用 MSVC 的 `std::random_device` (确定性的)。

非对称加密 API

RSA 2048 / 4096 ([PKCS#1 v2.2 / RFC 8017](#))

头文件: [include/rsa.hpp](#)

```
#include "rsa.hpp"

// 2048-bit
rsa_public_key pub; rsa_private_key prv;
rsa_keygen(pub, prv);

rsa_encrypt(pub, plaintext, ct);
rsa_decrypt(prv, ct, recovered);

// 4096-bit
rsa4096_bignum a = rsa4096_bignum::from_uint64(42);
rsa4096_public_key pub4; rsa4096_private_key prv4;
rsa4096_keygen(pub4, prv4);
```

支持的填充方案 ([include/rsa.hpp](#) 相关函数) :

- PKCS#1 v1.5 (RSA 加密 / 签名, [rsa_schemes.cpp](#))
- OAEP ([RFC 8017](#), [rsa_oaep.cpp](#))
- PSS ([RFC 8017](#), [rsa_pss.cpp](#))

性能特性:

- **Montgomery CIOS 乘法**, Windows 走 MASM MULX 汇编 ([rsa_mont_asm_win.asm](#)) , Linux 走 GCC 内联汇编 ([rsa_mont_asm.cpp](#)) ; CPU 不支持 BMI2/ADX 时自动回退标量。
- **CRT 半尺寸汇编**: 私钥解密 p/q 模幂使用半尺寸 Montgomery 乘法 ([mont_mul_half_asm](#)) , 核心吞吐提升约 1.3–1.45×。
- **CRT 私钥签名**: PKCS#1 完整私钥解析自动携带 CRT 参数 (p/q/dP/dQ/qInv) , [rsa_pkcs1_sign](#) 优先 RSASP1 两路半宽模幂 (RSA-2048 私钥签名 2186ms → 3ms、PSS 2436ms → 1ms) ; 仅含 n/d/e 的私钥自动回退全模幂。
- **OpenMP 多线程**: [rsa_decrypt](#) / [rsa_crt_decrypt](#) 的两路独立模幂 m1/m2 用 [#pragma omp parallel sections num_threads\(2\)](#) 并行 (2048/4096 解密实测提速 1.5–1.7×) ; [-DJP_ENABLE_OPENMP=OFF](#) 时自动回退串行。
- **批量模幂**: AVX2 / AVX-512 批量 Montgomery ([rsa_batch_dispatch.cpp](#)) , 用于服务器批量解密场景。
- **keygen 素数预算兜底**: 素数搜索预算 100ms, 超时后从预制素数表 (1024 位 ×50 对 + 2048 位 ×50 对, MR 已验证) 随机取一组完成 keygen, 保证永不卡死。预制素数公开、仅作参考/兜底, 不用于生产密钥。

ECDSA P-256 ([FIPS 186-4](#))

头文件: [include/ecdsa.hpp](#)

```
#include "ecdsa.hpp"

uint8_t pub[64], priv[32];
ecdsa_p256_keygen(pub, priv);

uint8_t sig[64];
ecdsa_p256_sign(priv, (const uint8_t*)"message", 7, sig);
```

```
bool ok = ecdsa_p256_verify(pub, (const uint8_t*)"message", 7, sig);
```

曲线: [secp256r1](#) (P-256, NIST 曲线)。

Ed25519 ([RFC 8032](#))

头文件: [include/ed25519.hpp](#) (批量验签: [include/ed25519_batch.hpp](#))

```
#include "ed25519.hpp"
```

```
uint8_t pub[32], priv[64];  
ed25519_keygen(pub, priv);
```

```
uint8_t sig[64];  
ed25519_sign(priv, (const uint8_t*)"message", 7, sig);
```

```
bool ok = ed25519_verify(pub, (const uint8_t*)"message", 7, sig);
```

加速: AVX-512 ([ed25519_avx512.cpp](#))、Radix-51 表示 ([ed25519_r51.cpp](#))、批量验签 ([ed25519_batch_dispatch.cpp](#))。

Ed448 ([RFC 8032](#))

头文件: [include/ed448.hpp](#) (批量验签: [include/ed448_batch.hpp](#))

```
#include "ed448.hpp"
```

```
uint8_t pub[57], priv[114];  
ed448_generate_keypair(pub, priv); // priv = seed(57) || pub(57)
```

```
uint8_t sig[114];  
ed448_sign(priv, (const uint8_t*)"message", 7, sig);
```

```
bool ok = ed448_verify(pub, (const uint8_t*)"message", 7, sig);
```

加速: AVX2 / AVX-512 ([ed448_avx2.cpp](#) / [ed448_avx512.cpp](#))、批量验签 ([ed448_batch_dispatch.cpp](#))。

X25519 ([RFC 7748](#))

头文件: [include/x25519.hpp](#)

```
#include "x25519.hpp"
```

```
uint8_t alice_pub[32], alice_priv[32];  
uint8_t bob_pub[32], bob_priv[32];  
x25519_generate_keypair(alice_pub, alice_priv);  
x25519_generate_keypair(bob_pub, bob_priv);
```

```
uint8_t shared[32];  
x25519_scalar_mult(shared, alice_priv, bob_pub);
```

加速: AVX-512 ([x25519_avx512.cpp](#))。

X448 ([RFC 7748](#))

头文件: [include/x448.hpp](#)

```
#include "x448.hpp"
```

```
uint8_t pub[56], priv[56];  
x448_generate_keypair(pub, priv);
```



```
uint8_t shared[56];
x448_scalar_mult(shared, priv, peer_pub);
```

加速：AVX2 / AVX-512 ([x448_avx2.cpp](#) / [x448_avx512.cpp](#)) ， 批量 ([test_x448_batch](#)) 。

SM2 ([GM/T 0003-2012](#), [RFC 8998](#) 国密 TLS)

头文件：[include/sm2.hpp](#)

```
#include "sm2.hpp"

uint8_t pub[64], priv[32];
sm2_keygen(pub, priv);

uint8_t sig[64];
sm2_sign(priv, (const uint8_t*)"message", 7, sig);
bool ok = sm2_verify(pub, (const uint8_t*)"message", 7, sig);

// 从私钥派生公钥
sm2_pub_from_priv(priv, pub);

// 计算用户标识杂凑值 ZA（默认 ID "1234567812345678"）
uint8_t za[32];
sm2_compute_za((const uint8_t*)"1234567812345678", 16, pub, pub+32, za);
```

曲线：sm2p256v1。性能特性：

- 域运算 Montgomery (CIOS) + wNAF-5 标量乘 + Shamir 双标量乘 + 批量仿射化
- keygen / sign / verify 相对旧实现提速 84× / 83× / 135×，对 OpenSSL 4.0 反超 1.31× / 1.35× / 1.26×
- 支持 OpenSSL 双向签名互操作验证 ([bench_sm_ossl](#) / [test_sm](#))

密钥与签名互操作

库提供与 OpenSSL 的交叉验证测试（安装 OpenSSL 后自动启用）：RSA、ECDSA、Ed25519、Ed448、X25519、X448、SM2、SM3、SM4、SHA-3、GCM 等均有对应对比测试（见 [测试与 CI](#)）。

X.509 v3 证书 API ([RFC 5280](#))

头文件：[include/x509.hpp](#)，命名空间 `jpssl::x509`。

提供完整的 X.509 v3 证书 **DER 编解码**、**自签名证书生成**和**证书链验证**。支持 RSA-2048/4096、Ed25519、Ed448、ECDSA P-256、SM2 五种密钥类型。

1. 生成自签名证书

```
#include "x509.hpp"
using namespace jpssl::x509;

// 生成 Ed25519 密钥
uint8_t pub[32], priv[64];
ed25519_keygen(pub, priv);

// 构建并签名自签名证书
x509_builder builder;
DistinguishedName dn;
dn.push_back({std::vector<uint8_t>(OID_CN, OID_CN + 3), "example.com"});
builder.set_subject(dn).set_issuer(dn);          // 自签名: subject == issuer

uint8_t serial[8] = {0x01, 0x02, 0x03, 0x04};
builder.set_serial(serial, 8);

uint64_t now = (uint64_t)time(nullptr);
builder.set_validity(now, now + 365ULL * 86400);    // 有效期 365 天

builder.set_key(KeyType::Ed25519, pub, 32);        // 公钥
builder.set_ca(false);                             // 叶子证书 (CA=false)
builder.set_key_usage(KU_DIGITAL_SIGNATURE);       // KeyUsage 扩展
builder.set_server_auth();                          // EKU: serverAuth
builder.add_san_dns("example.com");                // SAN: DNS 名称
builder.add_san_dns("www.example.com");

auto cert = builder.build_and_sign(KeyType::Ed25519, priv, 64); // 私钥签名

// 编码为 DER 字节
std::vector<uint8_t> der = cert.to_der();
```

2. 解析 DER / PEM 证书

```
// 从 DER 字节解析
auto parsed = x509_cert::from_der(der);
// 从 PEM 文本解析 (-----BEGIN CERTIFICATE-----)
auto parsed = x509_cert::from_pem(pem_string);
// 证书编码为 PEM
std::string pem = parsed->to_pem();

if (parsed) {
    std::string cn = parsed->common_name();    // 获取 subject CN
    std::string issuer = parsed->issuer_name(); // 获取 issuer CN
    bool is_ca = parsed->is_ca();              // 是否 CA 证书
    bool valid = parsed->is_valid_now();        // 是否在有效期内
    auto dns = parsed->dns_names();            // SAN DNS 名称列表
    KeyType kt = parsed->key_type();           // 密钥类型
}
```

`x509_cert` 还提供：

- **raw_extensions**：原始扩展保留，未知扩展在解析 / 编码时原样保留（证书透明的 SCT list / poison 扩展依赖此能力）

- 序列号、签名算法、SPKI、时间字段等访问器

3. 读取私钥 / CSR (PEM / DER)

```
// 私钥读取: 自动识别 PKCS#8 / PKCS#1 RSA / SEC1 EC / [RFC 8410](https://www.rfc-editor.org/rfc/rfc8410)
// (Ed25519/Ed448)
// -----BEGIN PRIVATE KEY----- / RSA PRIVATE KEY / EC PRIVATE KEY
// -----BEGIN ED25519 PRIVATE KEY----- / ED448 PRIVATE KEY-----
auto key = private_key::from_pem(key_pem);
if (key) {
    KeyType kt = key->key_type;    // 密钥类型
    auto& priv = key->priv;        // 私钥原始字节 (与 x509_builder::build_and_sign 兼容)
    auto& pub = key->pub;          // 公钥原始字节 (解析时从密钥中恢复)
}
// 支持 DER 输入: private_key::from_der(...)

// 加密私钥读取 (PBES2, -----BEGIN ENCRYPTED PRIVATE KEY-----)
// PBKDF2-HMAC-SHA256 + AES-128/256-CBC ([RFC 8018](https://www.rfc-editor.org/rfc/rfc8018))
auto enc_key = private_key::from_pem_encrypted(enc_pem, "password");

// CSR 读取 (PKCS#10, -----BEGIN CERTIFICATE REQUEST-----)
auto req = csr::from_pem(csr_pem);
if (req) {
    auto& subject = req->subject;    // DistinguishedName
    KeyType kt = req->key_type;      // 公钥类型
    auto& pub = req->public_key;     // 公钥原始字节
    auto& sig = req->signature;       // 签名原始字节
    auto& tbs = req->tbs_raw;         // CertificationRequestInfo 原始字节 (供验签)
}
```

私钥输出格式与库内 `x509_builder::build_and_sign` 完全兼容 (RSA=d、Ed25519=seed||pub、Ed448=seed、EC/SM2=scalar) , 解析出的密钥可直接用于证书签发。

4. 证书链验证

```
// 构建根 CA (自签名, CA=true, KeyCertSign)
x509_builder root_builder;
root_builder.set_ca(true, 0);          // pathLen=0
root_builder.set_key_usage(KU_KEY_CERT_SIGN);
auto root = root_builder.build_and_sign(KeyType::Ed25519, root_priv, 64);

// 叶子证书由根 CA 签发 (issuer=root, 用根私钥签名)
x509_builder leaf_builder;
leaf_builder.set_issuer(root_dn);
auto leaf = leaf_builder.build_and_sign(KeyType::Ed25519, root_priv, 64);

// 验证证书链: leaf → root
std::vector<x509_cert> chain = {leaf, root};
auto result = x509_verify_chain(chain, now);
if (result.success) {
    // 链有效: 签名正确、有效期未过期、根证书是 CA
} else {
    std::string err = result.error; // 失败原因
}
```

验证内容包括: 签名正确性、有效期、CA 属性 (BasicConstraints) 、KeyUsage (KeyCertSign) 、路径长度 (pathLen) 约束。

OpenSSL 互操作

证书签名验证兼容 OpenSSL 3.0 生成的证书:

- **签名格式**：同时接受定长 raw `r||s` 与本库自产格式，以及 DER 编码的 **ECDSA-Sig-Value** (`SEQUENCE { INTEGER r, INTEGER s }`)，OpenSSL 默认输出，支持长短格式长度）。
- **SM2**：验证时按国密标准计算 $e = \text{SM3}(\text{ZA} \parallel \text{TBS})$ （单层哈希），ZA 依次尝试空用户 ID（OpenSSL 3.0 x509 生成行为）与国密标准默认 ID `"1234567812345678"` ([GB/T 32918.5 / GM/T 0003](#))，兼容主流实现；本库签发的 SM2 证书与 OpenSSL 互验通过。
- **ECDSA P-256/P-384/P-521**：`ecdsa_pXXX_sign/verify` 内部自带哈希，签名 / 验证直接对 TBS 字节进行，避免双重哈希；P-384/P-521 曲线 OID 使用标准值 `1.3.132.0.34/35`。

双向互操作已实测：OpenSSL 签发的 SM2 / P-256 / P-384 / P-521 证书本库可验证，本库签发的证书 `openssl verify` 通过。

5. TLS 集成

```
#include "tls.hpp"
using namespace jpssl::tls;
```

```
// 从 tls_certificate 生成 X.509 DER 自签名证书
std::vector<uint8_t> der = tls_make_x509_self_signed(*tls_cert);
```

```
// 密钥类型映射
x509::KeyType kt = tls_sig_alg_to_key_type(cert->sig_alg);
```

TLS 握手时若 `tls_certificate::cert_data` 为空，会自动生成 X.509 v3 DER 证书并发送给对端（`tls13_make_certificate` / `tls12_make_certificate` 均支持）。

支持的关键类型

- **KeyType**：RSA2048、RSA4096、Ed25519、Ed448、ECDSA_P256、SM2
- **x509_builder**：链式设置 subject / issuer / serial / validity / key / CA / KeyUsage / EKU / SAN
- **x509_cert**：`from_der` / `from_pem` 解析、`to_der` / `to_pem` 编码、属性访问器
- **private_key**：`from_der` / `from_pem` / `from_pem_encrypted` (PKCS#8/PKCS#1/SEC1/[RFC 8410](#)/加密 PEM)
- **csr**：`from_der` / `from_pem` (PKCS#10)
- **x509_verify_chain**：证书链验证结果 { success, error }

命令行工具

也可以直接用 `jpssl-cert` 在命令行生成、查看和验证证书。

TLS API (TLS 1.2 / 1.3)

头文件：[include/tls.hpp](#)，命名空间 `jpssl::tls`。

TLS 模块提供完整的 TLS 1.2 和 TLS 1.3 握手流程、记录层加解密、SNI 多域名证书管理、0-RTT 早数据。支持 AES-128/256-GCM、ChaCha20-Poly1305、AES-128-CCM 等密码套件，以及 Ed25519、ECDSA P-256、RSA-2048/4096、SM2 等多种证书签名算法，并支持 [RFC 8998 国密套件](#) (TLS_SM4_GCM_SM3 + SM2)。

```
#include "tls.hpp"
using namespace jpssl::tls;
```

1. 创建证书

证书是 TLS 握手的关键组件，包含公钥、私钥和签名算法。jpssl 集成了 **X.509 v3** ([RFC 5280](#)) 证书编码：当 `cert_data` 为空时，握手过程会自动调用 `tls_make_x509_self_signed()` 生成标准的 X.509 v3 DER 自签名证书（含 SAN、KeyUsage、EKU 扩展），也可以手动预生成并填入 `cert_data`。

```
// — Ed25519 证书（握手时自动生成 X.509 DER） —
auto cert = std::make_unique<tls_certificate>();
cert->subject_name = "example.com";
cert->sig_alg = SignatureAlgorithm::ED25519;
ed25519_keygen(cert->pub.ed25519, cert->priv.ed25519);

// 可选：预生成 X.509 v3 DER 证书并填入 cert_data
cert->cert_data = tls_make_x509_self_signed(*cert);

// — ECDSA P-256 证书 —
auto ecdsa_cert = std::make_unique<tls_certificate>();
ecdsa_cert->subject_name = "example.org";
ecdsa_cert->sig_alg = SignatureAlgorithm::ECDSA_SECP256R1_SHA256;
ecdsa_p256_keygen(ecdsa_cert->pub.ecdsa_p256, ecdsa_cert->priv.ecdsa_p256);

// — RSA-2048 证书 —
auto rsa_cert = std::make_unique<tls_certificate>();
rsa_cert->subject_name = "example.net";
rsa_cert->sig_alg = SignatureAlgorithm::RSA_PKCS1_SHA256;
rsa_keygen(rsa_cert->pub.rsa, rsa_cert->priv.rsa);

// — SM2 证书 ([RFC 8998](https://www.rfc-editor.org/rfc/rfc8998) 国密 TLS) —
auto sm2_cert = std::make_unique<tls_certificate>();
sm2_cert->subject_name = "example.cn";
sm2_cert->sig_alg = SignatureAlgorithm::SM2_SM3;
sm2_keygen(sm2_cert->pub.sm2, sm2_cert->priv.sm2);
```

2. 多域名证书管理 (SNI)

证书管理器支持根据客户端 SNI 请求自动选择对应域名的证书，`get_certificate` 支持通配降级。

```
tls_certificate_manager cert_mgr;
cert_mgr.add_certificate("example.com", std::move(cert));
cert_mgr.add_certificate("example.org", std::move(ecdsa_cert));
cert_mgr.add_certificate("example.net", std::move(rsa_cert));

// 查询证书
const tls_certificate* c = cert_mgr.get_certificate("example.com");
if (c) { /* 使用证书 */ }

// 默认证书（第一个添加的证书）
const tls_certificate* def = cert_mgr.get_default_certificate();
```

3. TLS 1.3 完整握手（推荐）

TLS 1.3 使用 X25519 进行密钥交换。国密套件用法：设置 `client.cipher_suite = CipherSuite::TLS_SM4_GCM_SM3`，服务端使用 SM2 证书自动协商。

```
// — 服务端：准备证书管理器 —
tls_certificate_manager cert_mgr;
cert_mgr.add_certificate("example.com", std::move(server_cert));

// — 客户端：发起握手 —
tls_session client;
client.server_name = "example.com"; // SNI: 指定目标域名

// Step 1: 客户端生成 ClientHello
std::vector<uint8_t> client_hello;
tls13_make_client_hello(client, client_hello);

// 将 client_hello 通过网络发送给服务端...

// — 服务端：处理 ClientHello，生成完整回包 —
tls_session server;
std::vector<uint8_t> server_flight;
tls13_make_server_flight(server, client_hello.data(), client_hello.size(),
                        server_flight, cert_mgr);

// 将 server_flight 通过网络发送给客户端...

// — 客户端：处理服务端回包，生成 Client Finished —
std::vector<uint8_t> client_finished;
tls13_process_server_flight(client, server_flight.data(), server_flight.size(),
                          client_finished, &cert_mgr);

// 将 client_finished 通过网络发送给服务端...

// — 服务端：验证客户端 Finished —
tls13_process_client_finished(server, client_finished.data(), client_finished.size());

// 握手完成！双向安全通信

// 客户端 → 服务端
auto client_record = tls_encrypt(client, ContentType::APPLICATION_DATA,
                                (const uint8_t*)"Hello, TLS 1.3!", 16);
ContentType ct; std::vector<uint8_t> plaintext;
tls_decrypt(server, client_record.data(), client_record.size(), ct, plaintext);

// 服务端 → 客户端（使用服务端专用 API）
auto server_record = tls_server_encrypt(server, ContentType::APPLICATION_DATA,
                                       (const uint8_t*)"Hi from server!", 15);
ContentType ct2; std::vector<uint8_t> resp;
tls_decrypt(client, server_record.data(), server_record.size(), ct2, resp);
```

4. TLS 1.3 简化版握手（一次性）

适合不需要分步处理握手消息的场景。注意简化版只交换 ServerHello，不包含加密握手消息。

```
tls_session client, server;
tls_certificate_manager cert_mgr;
cert_mgr.add_certificate("localhost", std::move(cert));

// 服务端处理 ClientHello 并生成 ServerHello
std::vector<uint8_t> sh, ch;
tls13_make_client_hello(client, ch);
tls13_handshake_server(server, ch.data(), ch.size(), sh, cert_mgr);

// 客户端处理服务端回包
std::vector<uint8_t> dummy;
```

```
tls13_process_server_flight(client, sh.data(), sh.size(), dummy, &cert_mgr);
```

```
// 安全通信（应用数据加解密）
auto enc = tls_encrypt(client, ContentType::APPLICATION_DATA,
    (const uint8_t*)"data", 4);
ContentType t; std::vector<uint8_t> dec;
tls_decrypt(server, enc.data(), enc.size(), t, dec);
```

5. TLS 1.3 0-RTT（零往返）+ PSK 会话恢复

0-RTT 允许客户端在握手完成前发送应用数据，基于前一次握手的 PSK（Pre-Shared Key）恢复会话。

```
// =====
// 阶段 1: 完整握手 + 获取 NewSessionTicket
// =====

tls_session client, server;
tls_certificate_manager cert_mgr;
cert_mgr.add_certificate("example.com", std::move(server_cert));

std::vector<uint8_t> ch, sf, cf;
tls13_make_client_hello(client, ch);
tls13_make_server_flight(server, ch.data(), ch.size(), sf, cert_mgr);
tls13_process_server_flight(client, sf.data(), sf.size(), cf, &cert_mgr);
tls13_process_client_finished(server, cf.data(), cf.size());

// =====
// 阶段 2: 服务端生成会话票据
// =====

std::vector<uint8_t> ticket_msg;
tls13_make_new_session_ticket(server, ticket_msg);

auto enc_ticket = tls_encrypt_handshake(server, ticket_msg.data(), ticket_msg.size());
// 客户端接收并存储 PSK
tls13_store_psk(client, ticket_msg.data(), ticket_msg.size());

// =====
// 阶段 3: 后续连接 — PSK 恢复 + 0-RTT 早数据
// =====

tls_session client2, server2;
// 客户端复制 PSK（实际应用中从持久化存储加载）
client2.psk_valid = true;
memcpy(client2.psk_identity, client.psk_identity, client.psk_identity_len);
client2.psk_identity_len = client.psk_identity_len;
memcpy(client2.psk_value, client.psk_value, tls_hash_len(client.cipher_suite));
client2.ticket_age_add = client.ticket_age_add;
client2.ticket_issue_time = client.ticket_issue_time;
client2.server_name = "example.com";

// 服务端也要有相同的 PSK（实际应用中从数据库/缓存加载）
server2.psk_valid = true;
memcpy(server2.psk_identity, server.psk_identity, server.psk_identity_len);
server2.psk_identity_len = server.psk_identity_len;
memcpy(server2.psk_value, server.psk_value, tls_hash_len(server.cipher_suite));
server2.is_server = true;

// 客户端生成含 PSK 扩展的 ClientHello
std::vector<uint8_t> psk_ch;
tls13_make_psk_client_hello(client2, psk_ch);

// 服务端处理 PSK ClientHello，接受 early_data
bool accept_early_data = false;
tls13_process_psk_client_hello(server2, psk_ch.data(), psk_ch.size(), accept_early_data);

// 客户端发送 0-RTT 早数据（在握手完成前！）
auto early_data = tls13_encrypt_early_data(client2,
    (const uint8_t*)"Early data before handshake!", 29);
```

```
// 服务端解密早数据
ContentType ct; std::vector<uint8_t> early_plain;
tls13_decrypt_early_data(server2, early_data.data(), early_data.size(), ct, early_plain);

// 服务端发送 EndOfEarlyData, 然后继续正常握手流程
auto eoed = tls13_make_end_of_early_data();
auto enc_eoed = tls_encrypt_handshake(server2, eoed.data(), eoed.size());
tls13_process_end_of_early_data(client2, eoed.data(), eoed.size());
```

6. TLS 1.2 握手 (RFC 5246)

支持 **RSA** 和 **ECDHE** (X25519) 两种密钥交换方式, 完整的密码套件协商、ServerKeyExchange、Certificate、ServerHelloDone 消息流程。

```
// — 服务端: 准备 RSA 证书 —
tls_certificate_manager cert_mgr;
auto rsa_cert = std::make_unique<tls_certificate>();
rsa_cert->sig_alg = SignatureAlgorithm::RSA_PKCS1_SHA256;
rsa_keygen(rsa_cert->pub.rsa, rsa_cert->priv.rsa);
cert_mgr.add_certificate("example.com", std::move(rsa_cert));

// — 客户端: 发起握手 —
tls_session client;
client.server_name = "example.com";

std::vector<uint8_t> ch12;
tls12_make_client_hello(client, ch12);

uint8_t pre_master[48];
for (int i = 0; i < 48; i++) pre_master[i] = (uint8_t)(rand() % 256);
pre_master[0] = 0x03; pre_master[1] = 0x03; // TLS 1.2 协议版本

uint8_t encrypted_pms[256];
rsa_encrypt(cert_mgr.get_certificate("example.com")->pub.rsa,
            std::span<const uint8_t>(pre_master, 48), encrypted_pms);

// — 服务端: 处理 ClientHello, RSA 解密 pre-master —
tls_session server;
uint8_t decrypted_pms[48];
std::vector<uint8_t> sh12;
tls12_make_server_flight(server, ch12.data(), ch12.size(), sh12,
                        encrypted_pms, 256, decrypted_pms, cert_mgr);

// — 客户端: 处理服务端回包, 生成 Finished —
std::vector<uint8_t> cf12;
tls12_process_server_flight(client, sh12.data(), sh12.size(),
                            pre_master, 48, cf12);

// — 服务端: 验证客户端 Finished —
tls12_process_client_finished(server, cf12.data(), cf12.size());

// 握手完成! 安全通信
auto enc = tls_encrypt(client, ContentType::APPLICATION_DATA,
                      (const uint8_t*)"TLS 1.2 data", 12);
ContentType t; std::vector<uint8_t> dec;
tls_decrypt(server, enc.data(), enc.size(), t, dec);
```

7. 记录层加解密 (握手完成后)

握手完成后, 所有应用数据通过记录层加密传输。tls_encrypt / tls_decrypt 根据 session.is_server 自动选择正确的密钥方向。

```
// — 通用 API (双向) —
std::vector<uint8_t> encrypted = tls_encrypt(
    session, ContentType::APPLICATION_DATA,
```



```

plaintext_data, plaintext_len);
ContentType ct; std::vector<uint8_t> decrypted;
bool ok = tls_decrypt(session, encrypted.data(), encrypted.size(), ct, decrypted);

// — 服务端专用 API (自文档化, 显式方向) —
auto server_record = tls_server_encrypt(server, ContentType::APPLICATION_DATA,
                                         (const uint8_t*)"response", 8);
ContentType sct; std::vector<uint8_t> from_client;
bool s_ok = tls_server_decrypt(server, client_record.data(),
                               client_record.size(), sct, from_client);

```

8. 大消息自动分片与合并

- **记录层** (`tls_encrypt` / `tls_decrypt`) : 明文 > 16KiB (`TLS_MAX_RECORD_PLAINTEXT`) 自动拆分为多条 ≤16KiB 的 record (TLS 1.2 / 1.3 均支持), 解密时逐条解析并自动合并还原, 单次调用即可收发任意大小的消息。
- **socket 层** (`tls_connection::send` / `recv`) : 一次 `send`(大缓冲) 自动分片写出; `recv`() 会把同一突发到达的多条 record 合并后一次返回, 大消息无需循环读取。消息边界由应用层协议负责 (如 HTTP Content-Length) 。
- 测试 `test_tls_large_msg`: 覆盖 16KiB 边界、64KiB 长度字段边界、256KiB、TLS 1.2 以及 socket 端到端 128KiB 单次 `send` / 单次 `recv`。

9. 握手 API 总览

函数	说明
<code>tls13_make_client_hello(s, out)</code>	客户端生成 ClientHello (含 SNI + X25519 key_share)
<code>tls13_make_server_flight(s, ch, len, out, cert_mgr)</code>	服务端处理 ClientHello, 生成完整回包 (SH+EE+Cert+CV+SF)
<code>tls13_process_server_flight(s, data, len, out, cert_mgr, trust)</code>	客户端处理服务端回包, 生成 Client Finished; <code>trust</code> (<code>tls_trust_store*</code>) 提供时对服务端证书链执行 x509 验证 (含主机名匹配), 失败则握手失败
<code>tls13_process_client_finished(s, data, len)</code>	服务端验证客户端 Finished
<code>tls13_handshake_client(s, ch, resp, len)</code>	简化版客户端握手 (一次性)
<code>tls13_handshake_server(s, ch, len, out, cert_mgr)</code>	简化版服务端握手 (一次性)
<code>tls13_make_new_session_ticket(s, out, lifetime)</code>	服务端生成 NewSessionTicket (握手后调用)
<code>tls13_store_psk(s, ticket, len)</code>	客户端从票据中提取并存储 PSK
<code>tls13_make_psk_client_hello(s, out)</code>	客户端生成含 PSK 扩展的 ClientHello (0-RTT)
<code>tls13_process_psk_client_hello(s, ch, len, accept)</code>	服务端处理 PSK ClientHello, 验证 binder 和票据
<code>tls13_encrypt_early_data(s, data, len)</code>	客户端加密 0-RTT 早数据
<code>tls13_decrypt_early_data(s, rec, len, ct, out)</code>	服务端解密 0-RTT 早数据
<code>tls13_make_end_of_early_data()</code>	服务端生成 EndOfEarlyData 消息
<code>tls13_process_end_of_early_data(s, data, len)</code>	客户端处理 EndOfEarlyData
<code>tls12_make_client_hello(s, out)</code>	客户端生成 TLS 1.2 ClientHello (含密码套件协商 + signature_algorithms)

函数	说明
<code>tls12_make_server_flight(s, ch, len, out, epms, eplen, pms, cert_mgr)</code>	服务端处理 ClientHello，密码套件协商，RSA 解密或 ECDHE 密钥交换，生成完整回包
<code>tls12_process_server_flight(s, resp, len, pms, pms_len, out)</code>	客户端处理服务端回包（SH+Cert+SKX+SHD），生成 Finished
<code>tls12_process_client_finished(s, data, len)</code>	服务端验证客户端 Finished
<code>tls12_make_certificate(cert)</code>	构造 TLS 1.2 Certificate 消息（自动生成 X.509 DER）
<code>tls_make_x509_self_signed(cert, days)</code>	从 <code>tls_certificate</code> 生成 X.509 v3 DER 自签名证书
<code>tls_sig_alg_to_key_type(sig_alg)</code>	将 SignatureAlgorithm 映射为 X.509 KeyType
<code>tls12_make_server_hello_done()</code>	构造 ServerHelloDone 消息
<code>tls12_make_client_key_exchange(pub, pms)</code>	构造 ClientKeyExchange（RSA 加密 pre-master）
<code>tls12_make_finished(s, for_server)</code>	构造 TLS 1.2 Finished 消息
<code>tls12_verify_finished(s, data, len, for_server)</code>	验证 TLS 1.2 Finished
<code>tls_make_change_cipher_spec()</code>	构造 ChangeCipherSpec 记录
<code>tls_make_alert(level, desc)</code>	构造 Alert 记录
<code>tls_encrypt(s, ct, data, len)</code>	记录层加密
<code>tls_decrypt(s, record, len, ct, out)</code>	记录层解密
<code>tls_encrypt_handshake(s, hs_msg, len)</code>	加密握手消息（TLS 1.3 内部）
<code>tls_server_encrypt(s, ct, data, len)</code>	服务端加密发送（等价于 <code>tls_encrypt</code> ，显式方向）
<code>tls_server_decrypt(s, rec, len, ct, out)</code>	服务端解密客户端数据（等价于 <code>tls_decrypt</code> ）
<code>tls_server_encrypt_handshake(s, hs, len)</code>	服务端加密握手消息（内部使用）

10. 证书管理 API

类 / 函数	说明
<code>tls_certificate</code>	证书结构体，含公钥、私钥、签名算法
<code>tls_certificate::sign(data, len, sig, sig_len)</code>	使用证书私钥签名
<code>tls_certificate::verify(data, len, sig, sig_len)</code>	使用证书公钥验证签名
<code>tls_certificate::from_pem(cert_pem, key_pem, err)</code>	从 PEM 证书 + PEM 私钥构造服务端证书（私钥支持 PKCS#8/PKCS#1/SEC1/ RFC 8410 /加密 PEM）
<code>tls_certificate::from_pem_file(cert_path, key_path, err)</code>	从证书 / 私钥 PEM 文件构造
<code>tls_certificate::from_csr_pem(csr_pem, key_pem, err)</code>	从 CSR + 私钥构造（握手时按 CSR 主体自动生成自签证书）
<code>tls_certificate::from_csr_pem_file(csr_path, key_path, err)</code>	从 CSR / 私钥 PEM 文件构造
<code>tls_certificate_manager::add_certificate(domain, cert)</code>	添加域名对应的证书
<code>tls_certificate_manager::get_certificate(domain)</code>	根据域名获取证书（支持通配降级）
<code>tls_certificate_manager::get_default_certificate()</code>	获取默认证书
<code>tls_trust_store</code>	客户端信任库：CA 根证书集合（ <code>from_pem</code> / <code>from_pem_file</code> / <code>from_system</code> 系统信任库）

类 / 函数	说明
<code>tls_key_type_to_sig_alg(kt)</code>	KeyType → TLS 签名方案映射 (from_pem 内部使用)
<code>tls_parse_server_name(extensions, len)</code>	从扩展中解析 SNI 域名

11. 会话状态

字段	说明
<code>tls_session::ver</code>	TLS 版本 (V12 / V13)
<code>tls_session::server_name</code>	SNI 客户端请求的域名
<code>tls_session::client_random / server_random</code>	32 字节随机数
<code>tls_session::cipher_suite</code>	协商的密码套件 (AES-128/256-GCM, ChaCha20, CCM, SM4-GCM/CCM)
<code>tls_session::handshake_secret / master_secret</code>	握手机密 / 主密钥 (32 或 48 字节, 取决于套件)
<code>tls_session::client_write_key / server_write_key</code>	记录层加密密钥 (16 或 32 字节, 取决于套件)
<code>tls_session::client_write_iv / server_write_iv</code>	记录层加密 IV (12 字节)
<code>tls_session::client_seq / server_seq</code>	记录层序列号 (防重放)
<code>tls_session::is_server</code>	是否为服务端会话
<code>tls_session::transcript_hash</code>	握手 transcript 哈希 (32 或 48 字节)
<code>tls_session::psk_valid / psk_identity / psk_value</code>	PSK 恢复会话状态 (0-RTT)
<code>tls_session::client_early_write_key / client_early_write_iv</code>	0-RTT 早数据加密密钥和 IV
<code>tls_session::early_data_accepted</code>	服务端是否接受了 0-RTT 早数据

TLS socket 封装层

头文件：[include/tls_socket.hpp](#)，命名空间 `jpssl::tls`。

在消息级 TLS API ([tls.hpp](#)) 之上提供跨平台 (Windows Winsock / Linux POSIX) 的 socket 封装 (TLS 1.3, [RFC 8446](#))：TCP 流式连接 + UDP 数据报，并支持托管调用方已创建的外部 fd。

- `tls_connection`：客户端 `connect(host, port, trust_store)` 或服务端 `server_handshake(cert_manager)` 完成 TLS 1.3 握手；`send / recv` 收发加密应用数据。
- 客户端默认只信任系统信任库：不传 `trust_store` (或传 `nullptr`) 时自动通过 `tls_trust_store::from_system()` 加载系统 CA bundle，对服务端证书链执行 x509 验证 (含主机名匹配)，验证失败或系统信任库不可用则握手失败。
- `tls_listener`：`listen(port) + accept(conn, cert_manager)`，接受连接并自动完成服务端握手。
- record 层自动处理半包 / 粘包、握手消息封装与加密 record 透传。

服务端示例

```
#include "tls_socket.hpp"
using namespace jpssl::tls;

// 准备证书管理器
tls_certificate_manager cert_mgr;
// ... add_certificate("example.com", ...) ...

tls_listener listener;
if (!listener.listen(443, "0.0.0.0")) {
    // 绑定失败
}

tls_connection conn;
if (listener.accept(conn, cert_mgr)) { // accept 内自动完成 TLS 1.3 服务端握手
    std::vector<uint8_t> req;
    if (conn.recv(req)) {
        conn.send("HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nok");
    }
}
```

客户端示例

```
tls_connection conn;
// 默认只信任系统信任库 (无需显式 trust_store) :
// 自动加载系统 CA bundle 并验证服务端证书链
if (conn.connect("example.com", 443)) {
    conn.send("GET / HTTP/1.1\r\nHost: example.com\r\n\r\n");

    std::vector<uint8_t> resp;
    if (conn.recv(resp)) {
        // resp 为完整应用消息 (多条 record 已自动合并)
    }
}

// 也可显式指定自定义 CA 根:
tls_trust_store trust = tls_trust_store::from_pem_file("ca.crt");
tls_connection conn2;
if (conn2.connect("example.com", 443, trust)) { /* ... */ }
```

成员一览

成员	说明
<code>tls_socket_init(error)</code>	初始化 socket 子系统（Windows 下执行一次 WSAStartup；POSIX 为空操作）， <code>connect</code> / <code>listen</code> 内部会自动调用
<code>tls_connection::connect(host, port, trust_store, error)</code>	客户端：TCP 连接 + TLS 1.3 握手。默认（ <code>trust_store == nullptr</code> ）只信任系统信任库（ <code>tls_trust_store::from_system()</code> ）；传 <code>tls_certificate_manager*</code> 保持按 SNI 查找预期证书的旧行为
<code>tls_connection::connect(host, port, trust, error)</code>	客户端：TCP 连接 + TLS 1.3 握手，按 <code>tls_trust_store&</code> （自定义 CA 根）验证服务端证书链
<code>tls_connection::server_handshake(cert_manager, error)</code>	服务端：对已建立的 TCP 连接执行 TLS 1.3 握手（通常由 <code>accept</code> 调用）
<code>tls_connection::send(data, len, error)</code>	发送应用数据，自动按 ≤16KiB 分片为多条 TLS record
<code>tls_connection::recv(out, error)</code>	读取应用数据，同一突发到达的多条 record 自动合并后一次返回
<code>tls_connection::close() / is_open()</code>	关闭连接 / 连接状态
<code>tls_connection::session()</code>	访问底层 <code>tls_session</code> （套件、密钥、序列号等）
<code>tls_connection::native()</code>	原生 socket 句柄
<code>tls_connection::attach(fd, take_ownership, error)</code>	托管外部 socket 句柄（TCP 已连接 / <code>accept</code> 出的连接 / UDP 已 <code>connect</code> 或已 <code>bind</code> ）； <code>take_ownership=false</code> 借用模式： <code>close()</code> 不关闭、不 shutdown 外部句柄
<code>tls_connection::client_handshake(host, trust, error)</code>	在已托管 socket（ <code>attach</code> 之后）上执行客户端握手，不建立传输连接；信任语义与 <code>connect</code> 一致
<code>tls_connection::owns_socket()</code>	是否持有外部句柄所有权（ <code>close()</code> 时是否关闭底层 fd）
<code>tls_connection::set_datagram_mode(enable, error)</code>	数据报模式（UDP 链接）开关； <code>attach</code> 对 <code>SOCK_DGRAM</code> 自动启用， <code>enable=true</code> 要求底层为 UDP socket
<code>tls_connection::is_datagram()</code>	是否数据报模式
<code>tls_listener::attach(fd, take_ownership, error)</code>	托管外部监听 socket（TCP 已 <code>listen</code> / UDP 已 <code>bind</code> ，自动检测）
<code>tls_listener::owns_socket()</code>	是否持有外部句柄所有权
<code>tls_listener::is_udp()</code>	是否 UDP 监听器
<code>tls_listener::listen_udp(port, addr, error)</code>	UDP 链接：绑定 UDP 端口等待客户端握手（数据报模式）
<code>tls_listener::accept_udp(conn, cert_mgr, error)</code>	接收一个 UDP 客户端并完成服务端握手（监听 socket 转交给 <code>conn</code> ）
<code>tls_listener::listen(port, bind_addr, error)</code>	监听端口（默认 0.0.0.0）
<code>tls_listener::accept(conn, cert_manager, error)</code>	接受连接并自动完成服务端握手
<code>tls_listener::close()</code>	关闭监听 socket
<code>tls_connection::set_nonblocking(enable, error)</code>	设置/取消非阻塞模式（可在 <code>connect</code> 之前调用）；非阻塞下 <code>send/recv</code> 遇 EAGAIN 立即返回 <code>false + would-block</code>
<code>tls_connection::would_block()</code>	最近一次 I/O 是否因资源暂不可用而返回（仅非阻塞模式）
<code>tls_connection::wait_readable(ms) / wait_writable(ms)</code>	等待 socket 可读 / 可写（事件循环用， <code>ms < 0</code> 无限等待）
<code>tls_connection::set_handshake_timeout(ms)</code>	握手阶段有界等待超时（默认 30000 ms）
<code>tls_listener::set_nonblocking(enable, error)</code>	监听 socket 非阻塞； <code>accept</code> 出的连接继承非阻塞状态

成员	说明
<code>tls_connection::attach_co_executor(ex)</code>	绑定协程执行器（ <code>co_send/co_recv</code> 前置）
<code>tls_connection::co_send(data, len, error)</code>	协程发送：写缓冲暂满时协程挂起，可写后由执行器恢复
<code>tls_connection::co_recv(out, error)</code>	协程接收：语义与 <code>recv()</code> 一致，无数据时挂起等待可读
<code>tls_co_executor::run_once(ms) / run(ms)</code>	驱动协程执行器：poll 就绪 + 恢复挂起协程

外部 fd 托管

`tls_connection` / `tls_listener` 均可托管调用方已创建的外部 socket 句柄（事件循环 / epoll / 自建连接等场景），并可选择是否持有句柄所有权：

```
// 客户端：手动建立 TCP 连接后托管，再在已有 socket 上做 TLS 握手
int fd = socket(AF_INET, SOCK_STREAM, 0);
connect(fd, ...);           // 调用方自己建连
tls_connection conn;
conn.attach(fd, /*take_ownership=*/true, &err);
conn.client_handshake("example.com", &trust, &err); // 不重建 TCP
conn.send("GET / HTTP/1.1\r\n\r\n", &err);
```

```
// 借用模式：close() 不关闭外部 fd，生命周期由调用方管理
tls_connection borrowed;
borrowed.attach(fd, false, &err);    // owns_socket() == false
borrowed.server_handshake(cert_mgr, &err);
borrowed.close();                   // fd 仍有效，调用方自行释放
```

UDP 链接（数据报模式）—— ❌❌ 非标准，存在缺陷，仅限自研两端互通

`tls_connection` 支持在 UDP 上承载 TLS（数据报模式）：每条 TLS record（含握手消息）封装为一个 UDP 数据报发送——UDP 发送是原子的，整包成功或失败；`send()` 对大消息自动分片为多个数据报，`recv()` 自动合并还原。

```
// 服务端：绑定 UDP 端口，接收首个 ClientHello 后固定对端并完成握手
tls_listener udp_listener;
udp_listener.listen_udp(8443, "0.0.0.0", &err);
tls_connection conn;
udp_listener.accept_udp(conn, cert_mgr, &err); // 监听 socket 转交给 conn
conn.send("hello over udp", &err);
```

```
// 客户端：手动创建已 connect 的 UDP socket 后托管
int ufd = socket(AF_INET, SOCK_DGRAM, 0);
connect(ufd, ...);           // 固定服务端地址
tls_connection client;
client.attach(ufd, true, &err);    // 自动检测 SOCK_DGRAM -> 数据报模式
client.client_handshake("example.com", &trust, &err);
std::vector<uint8_t> resp;
client.recv(resp, &err);
```

❌❌ 已知缺陷（与标准 DTLS / QUIC 不互通）

本数据报模式是自研的简化封装，不是标准 DTLS，也不是 QUIC：

- **非标准协议，无互操作性**：报文格式与 DTLS ([RFC 6347](#) / [RFC 9147](#)) 和 QUIC ([RFC 9000](#)) 完全不同，无法与 OpenSSL、Wireshark、浏览器等标准实现互通，仅限本库客户端与服务端之间使用。
- **无抗 DoS 机制**：缺少 DTLS 的 `HelloVerifyRequest` 无状态 cookie，服务端对任意伪造源地址的 `ClientHello` 都会建立会话状态。

- **无握手分片 / 重传 / 乱序重组**: 握手消息不按 `message_seq` 分片编号, 无 flight 超时重传; UDP 丢包 (尤其大证书链跨多个数据报时) 直接导致握手失败, 由调用方重试整个握手。
- **无防重放**: 应用数据记录不带 epoch/sequence 滑动窗口, 重放的旧数据报会被当作新数据接受。

未来计划: 后续版本将实现标准 **DTLS 1.2/1.3** ([RFC 6347](#) / [RFC 9147](#), 含 cookie、握手分片/重传/乱序重组、防重放) 以及 **QUIC + HTTP/3** ([RFC 9000](#) / [RFC 9114](#)), 届时本数据报模式将被标准协议取代。在此之前, UDP 场景请评估上述缺陷, 并优先考虑使用 TCP + TLS 或标准 DTLS 实现。

非阻塞模式 (事件循环)

`tls_connection` / `tls_listener` 支持非阻塞模式, 便于单线程事件循环复用同一线程服务大量连接:

```
tls_listener listener;
listener.listen(443, "0.0.0.0");
listener.set_nonblocking(true);    // 无连接时 accept 返回 would-block

for (;;) {
    tls_connection conn;
    if (!listener.accept(conn, cert_mgr)) {
        if (listener.would_block()) { listener.wait_readable(100); continue; }
        break;    // 真实错误
    }
    // accept 出的连接继承非阻塞状态
    std::vector<uint8_t> msg;
    if (!conn.recv(msg)) {
        if (conn.would_block()) { conn.wait_readable(100); continue; }
        break;
    }
    if (!conn.send("HTTP/1.1 200 OK\r\n\r\n")) {
        if (conn.would_block()) conn.wait_writable(100);
    }
}
```

- `set_nonblocking(true)` 可在 `connect` / `listen` 之前调用; 非阻塞 `connect` 走 `EINPROGRESS` + `poll` 等待可写 + `SO_ERROR` 检查。
- `send()` / `recv()` 遇 `EAGAIN/EWOULDBLOCK` 立即返回 `false` (连接保持打开), 配合 `would_block()` + `wait_readable()` / `wait_writable()` 重试。
- 握手阶段为有界等待 (`set_handshake_timeout`, 默认 30 秒), 不会永久阻塞。

协程 I/O (C++20)

应用数据收发可写成协程: `co_send` / `co_recv` 在 socket 暂不可用时挂起, 由执行器 `tls_co_executor` 在可读 / 可写时恢复, 不阻塞任何线程:

```
tls_co_executor ex;    // 单线程 poll 驱动执行器, 多连接共享

tls_connection conn;
conn.set_nonblocking(true);
conn.attach_co_executor(&ex);

tls_co_task<void> session() {    // 顶层协程任务 (由持有者析构清理)
    bool ok = co_await conn.co_send("GET / HTTP/1.1\r\n\r\n");
    std::vector<uint8_t> resp;
    co_await conn.co_recv(resp);
}

auto task = session();
ex.run();    // 驱动: poll 就绪并恢复挂起协程
```

- `tls_co_task<T>`: 泛型协程任务 (热启动 + 对称转换, 零外部依赖)。

- `tls_co_executor`: 单线程 poll 驱动执行器, 多个连接共享; `run_once()` / `run()` 在 socket 就绪时恢复挂起协程。
- `co_send` / `co_recv` 语义与 `send` / `recv` 一致 (自动分片/合并 record、跳过 `NewSessionTicket`) ; 使用前需 `set_nonblocking(true)` 并 `attach_co_executor(&ex)`。
- 可选增强: 通过绑定的协程线程池 (需 `threadpool` submodule) 可使协程体并行运行在线程池工作线程上, 相关内容在独立功能分支维护。

客户端验证

- **默认 (推荐)**: 只信任系统信任库中的 CA 根证书 (`tls_trust_store::from_system()`, 自动探测 `SSL_CERT_FILE` 环境变量与常见系统路径) , 对服务端证书链执行 x509 验证, 含叶子证书主机名匹配。
- **显式自定义信任库**: `tls_trust_store::from_pem` / `from_pem_file` 加载自管 CA 根后传入 `connect`。
- **兼容旧行为**: 传 `tls_certificate_manager*` 时按 SNI 名称查找预期服务器证书校验 `CertificateVerify`。

相关示例

- HTTPS + 证书透明示例
- [非阻塞 socket 示例](#): `nonblocking_echo` — 非阻塞监听 + 收发事件循环回环
- [协程 I/O 示例](#): `coroutine_echo` — 双端协程回环
: `examples/https/` 的服务器与客户端基于本封装层实现, 演示完整 HTTP 消息往返与 CT 审计端点。

证书透明 (Certificate Transparency)

头文件: `include/ct.hpp`, 源文件 `src/ct.cpp`。

JPSSL 提供两套证书透明实现:

- **国际标准**: 基于 [RFC 6962](#), SHA-256 默克尔树 + ECDSA P-256 (或 RSA-2048 PKCS#1 v1.5) 签名;
- **国密**: 基于 [RFC 6962](#) 框架、以 SM2/SM3 替代 ECDSA/SHA-256 (参考 GM/T 《证书透明规范》草案)。

核心概念与功能

默克尔树

- 叶哈希 / 节点哈希: `sm3_leaf_hash` / `sm3_node_hash` (SM3 变体), `sha256_leaf_hash` / `sha256_node_hash` (SHA-256 变体)
- 树根: `merkle_root(leaf_hashes)`
- 审计路径: `audit_path(leaf_index, tree_size) + verify_audit_path(...)`
- 一致性证明: `consistency_proof(first, second) + verify_consistency(first, second, ...)` (按 [RFC 9162](#) §2.1.4.2 实现)

TLS 风格编解码

- `pre_cert` / `merkle_tree_leaf` / SCT / STH 的序列化与反序列化 (`serialize_precert` / `deserialize_precert` 等)
- LogID: 国际算法 `compute_log_id_std` (SHA-256(SPKI)) ; 国密算法按草案计算
- X.509 集成: `precert poison` / SCT list 扩展、`finalize_precert`

内存日志

- `sm2_ct_log(priv, pub)`: 国密日志 (SM3 默克尔树 + SM2 签名)
- `ct_log(CtHashAlg::SHA256, CtSigAlg::ECDSA_P256, priv, pub)`: 国际日志 (SHA-256 默克尔树 + ECDSA P-256 签名)
- `ct_log(rsa_cert_key, rsa_public_key)`: RSA 日志 (SHA-256 + RSA-2048 PKCS#1 v1.5)

日志接口 ([RFC 6962](#) 风格) :

```
add_pre_chain / add_chain    // 提交证书或预证书
get_sth                    // 获取 Signed Tree Head
get_proof_by_hash          // 包含性证明
get_entries                // 按序号获取日志条目
get_sth_consistency        // 一致性证明
```

签发与验证

```
// 国际标准 (ECDSA P-256)
issue_sct_std(...) / verify_sct_std(...)
sign_sth_std(...) / verify_sth_std(...)

// 国密 (SM2 + SM3)
issue_sct(...) / verify_sct(...) // 或 sm2_ct_log 内置方法
sign_sth(...) / verify_sth(...)

// RSA
issue_sct_rsa(...) / verify_sct_rsa(...)
sign_sth_rsa(...) / verify_sth_rsa(...)
```

注意

DigitallySigned 中的算法字节当前为国密草案占位值 (**CT_HASH_ALG_SM3 = 0x04**、**CT_SIG_ALG_SM2 = 0x04**)，草案未定稿，与外部国密 CT 日志互操作前需核对字节值。

HTTPS + CT 示例

examples/https/ 提供 HTTPS 服务器与客户端示例，基于 TLS socket 封装层：

```
# 终端 1：启动 HTTPS + CT 服务器（默认 8443）
./https_server 8443

# 终端 2：客户端执行证书链 / SCT / STH / 包含性证明审计
./https_client 127.0.0.1 8443
```

服务器内嵌内存 CT 日志，为 ECDSA 服务器证书签发 SCT 并暴露 **/ct/cert**、**/ct/precert**、**/ct/ca**、**/ct/log-key**、**/ct/sth**、**/ct/proof** 等审计端点；客户端通过 TLS 1.3 连接完成全部 CT 校验。

MUSA GPU 加速（实验性）

JPSSL 支持可选的 **MUSA GPU 加速**（摩尔线程 MUSA 架构），默认关闭。该功能仅支持 Linux，Windows 上 **-DJP_ENABLE_MUSA=ON** 会被自动禁用并提示。

启用

需要 **MUSA SDK 4.3.0+**：

```
cmake -B build -DCMAKE_BUILD_TYPE=Release -DJP_ENABLE_MUSA=ON
cmake --build build
```

```
# 运行测试时指向 MUSA 运行库
LD_LIBRARY_PATH=/usr/local/musa/lib ctest --output-on-failure
```

构建集成：CMake 通过 `/usr/local/musa/cmake` 的 `find_package(MUSA)` 查找 SDK；MUSA 内核（.mu 文件）由 `mcc` 编译，`--offload-arch=mp_21 / mp_22 / mp_31`。

GPU 内核

文件	内容
src/aes_gpu.mu	AES ECB kernel
src/chacha20_gpu.mu	ChaCha20 keystream kernel
src/rsa_gpu.mu	RSA 批量模幂 kernel
src/sha512_gpu.mu	SHA-384/512 GPU kernel
src/aes_musa.cpp	AES 主机端封装
src/rsa_musa.cpp	RSA GPU 封装
src/sha512_musa.cpp	SHA-512 MUSA 封装

持久化池 API

为了避免每次调用重复分配 GPU 内存，库提供**持久化内存池**：

```
// AES
auto* pool = musa_aes_pool_create(ctx, 64*1024*1024);
musa_aes_pool_encrypt_ecb(pool, input, output, num_blocks);

// ChaCha20
auto* cc = musa_chacha20_pool_create(key, nonce);
musa_chacha20_pool_xor(cc, input, output, num_blocks, 0);

// RSA（批量解密）
auto* rsa = musa_rsa_pool_create(prv, 1024);
musa_rsa_batch_decrypt(rsa, ciphers, plains, count);
```

宏守卫

所有 GPU 基准段（`bench_sha512` / `bench_hardware_accel` 中的 `musa_*` 调用、`bench_rsa_gpu` 目标）都由 `JP_MUSA` 宏守卫：`JP_ENABLE_MUSA=OFF`（默认）时自动跳过 GPU 段，基准程序仍可正常编译运行。

性能数据

在 MTT S80 + i7-13700K 上的实测（详见 性能基准）：

算法	CPU	GPU	加速比
AES-128 ECB (16MB)	5.5 GB/s (AES-NI)	2.5 GB/s	—
ChaCha20 (16MB)	0.5 GB/s	1.1 GB/s	2.2×
RSA 2048 模幂	19 ms/op	36 ms/op (batch)	—
RSA 4096 模幂	0.5 ms (e=65537)	—	—

GPU 加速目前为**实验性**：AES-ECB 尚不如 AES-NI，ChaCha20 在 GPU 上有 2.2× 加速，RSA 批量模幂主要面向批量解密场景。

性能基准

基准程序位于 [benchmarks/](#)，通过 `JP_ENABLE_BENCH` 开关构建（默认 OFF）：

```
cmake -B build -DCMAKE_BUILD_TYPE=Release -DJP_ENABLE_BENCH=ON
cmake --build build
```

基准目标

目标	内容
bench_rsa_cpu_gpu	RSA CPU vs GPU vs OpenSSL 综合基准（2048/4096 私钥、2048 公钥）
bench_rsa_gpu	RSA-2048/4096 CPU 批量 vs MUSA GPU 批量（仅 <code>JP_ENABLE_MUSA=ON</code> 时构建）
bench_sha512	SHA-512 CPU vs SSE4.1 SIMD；GPU 单块/批量（仅 MUSA 构建）
bench_hardware_accel	AES / ChaCha20-Poly1305 / SHA-512 硬件加速路径对比（GPU 段仅 MUSA 构建）
bench_cipher_suites	TLS 密码套件性能
bench_sm4	SM4（ECB/GCM）vs OpenSSL
bench_base64	base64 编解码：标量 vs AVX2 vs AVX512 vs 自动分发
bench_sm_ossl	SM3 吞吐 + SM2 keygen/sign/verify vs OpenSSL
bench_ed25519_ossl	Ed25519 签名/验证 vs OpenSSL（仅找到 OpenSSL 时构建）
bench_ed448_x448_ossl	Ed448 / X448 vs OpenSSL（仅找到 OpenSSL 时构建）
bench_x25519_ossl	X25519 ECDH vs OpenSSL（仅找到 OpenSSL 时构建）

已公布的实测数据

平台：MTT S80（GPU）+ i7-13700K（CPU）

算法	CPU	GPU	加速比
AES-128 ECB（16MB）	5.5 GB/s（AES-NI）	2.5 GB/s	—
ChaCha20（16MB）	0.5 GB/s	1.1 GB/s	2.2×
RSA 2048 模幂	19 ms/op	36 ms/op（batch）	—
RSA 4096 模幂	0.5 ms（e=65537）	—	—

对 OpenSSL 的对比结论（来自提交历史 / README）

项目	结果
SM2 keygen / sign / verify	相对旧实现提速 84× / 83× / 135×；对 OpenSSL 4.0 反超 1.31× / 1.35× / 1.26×
SM3 压缩函数	8 轮展开 + 常量折叠，吞吐与 OpenSSL 持平；MSVC 汇编路径约 390 MB/s

项目	结果
SM4 ECB / GCM	吞吐均反超 OpenSSL 约 1.2–1.3×
SM4-GCM GHASH	PCLMULQDQ 快速路径
RSA CRT 解密	半尺寸汇编 + OpenMP 双线程，2048/4096 解密提速 1.5–1.7×
RSA CRT 私钥签名	PKCS#1 完整私钥自动携带 CRT 参数，RSA-2048 私钥签名 2186ms → 3ms、PSS 2436ms → 1ms (1.1.9)
Base64	AVX2 编码约 21–22 GB/s、解码约 17 GB/s (vs 标量编码约 3 GB/s、解码约 0.2 GB/s)

说明

- 所有 GPU 基准段由 `JP_MUSA` 宏守卫，MUSA 关闭时自动跳过，程序仍可编译运行。
- OpenSSL 对比基准仅在系统找到 OpenSSL 时构建。
- CI 中 Windows 的基准 job 只构建不运行 (`ctest -E bench`)，避免 CI 负载下的计时波动导致偶发失败。

测试与 CI

运行测试

构建后（见 快速开始），通过 CTest 运行全部单元测试：

```
# Linux
ctest --test-dir build --output-on-failure

# Windows
ctest --test-dir build-win --output-on-failure
```

未安装 OpenSSL 时，OpenSSL 对比测试自动跳过；安装后自动启用更多交叉验证。

单元测试目标

CTest 名称	覆盖内容
test_dtls	DTLS 1.2/1.3 (RFC 6347/9147) 单元测试（记录层、cookie、分片/重传、密钥调度、Finished）
test_dtls_openssl_interop	DTLS 1.2 与 OpenSSL 4 双向互通（ECDHE-ECDSA × AES-128-GCM / ChaCha20-Poly1305）
test_dtls13_openssl_interop	DTLS 1.3 与 OpenSSL 互通检测（OpenSSL 未实现 DTLS 1.3 时编译期 SKIP）
test_dtls_wolfssl_interop	DTLS 1.2/1.3 与 wolfSSL 5.9.2 双向互通（4 套件 × 2 方向 = 8 用例，需 JP_WOLFSSL_PREFIX）
test_aes	AES 全覆盖单元测试（OpenSSL 对比）
test_aes_modes	AES 全模式 × 全密钥长度（单块/ECB/CBC/GCM 软件 /auto/AVX2/AVX512/CCM，OpenSSL 交叉验证与篡改检测）
test_aes_ccm	AES-CCM 单元测试
test_gcm_simple	GCM 基础测试
test_sha1	SHA-1（NIST 向量、边界长度、增量更新、AVX2/AVX512 多缓冲交叉验证、批量分派、OpenSSL 对比）
test_sha3 / test_sha512	SHA-3、SHA-384/512（OpenSSL 对比）
test_tls	TLS 1.2/1.3 单元测试
test_tls448	TLS 1.3 X448/Ed448 握手测试（OpenSSL 对比）
test_tls_sm	TLS 国密套件（RFC 8998）测试
test_tls_socket	TLS socket 封装层回环测试
test_tls_stability	TLS 稳定性 / 稳压测试（反复握手 + 数据传输，泄漏启发式检测）
test_tls_large_msg	TLS 大消息自动分片 / 合并（16KiB 边界、64KiB 长度字段边界、256KiB、TLS 1.2、socket 端到端 128KiB）
test_x509	X.509 v3 证书单元测试
test_ct	证书透明（国密 + 国际）单元测试
test_base64	Base64（RFC 4648 向量、全长度随机往返、SIMD 交叉验证、非法输入，484 断言）

CTest 名称	覆盖内容
test_ed25519_rfc / test_ed448_rfc	RFC 8032 向量 (OpenSSL 对比)
test_ed25519_debug / test_ed448_debug	调试测试
test_ed25519_batch / test_ed448_batch	批量验签测试
test_x25519_rfc / test_x448_rfc / test_x448_batch	RFC 7748 向量与批量测试
test_x25519_ossll	X25519 OpenSSL 对比
test_ecdsa	ECDSA 单元测试 (OpenSSL 对比)
test_sm	国密 SM2/SM3/SM4 测试 (OpenSSL 对比)
test_ghash / test_ghash_debug	GHASH 测试 (OpenSSL 对比)
test_openssl_compare	OpenSSL 综合比较测试
test_verify_direct / test_ossll_verify	验证测试 (OpenSSL 对比)

DTLS 1.2 / 1.3 合规性测试

tests/test_dtls_wolfssl_interop.cpp (CTest 目标 test_dtls_wolfssl_interop) 使用 wolfSSL 作为独立对端, 验证 jpssl DTLS 1.2 (RFC 6347) 与 DTLS 1.3 (RFC 9147) 的互通合规性:

- 方向 A: jpssl DTLS 服务器 □ wolfSSL DTLS 客户端 (wolfSSL 校验 jpssl 证书链)
- 方向 B: wolfSSL DTLS 服务器 □ jpssl DTLS 客户端 (jpssl 校验 wolfSSL 证书链)
- 套件: DTLS 1.2 (ECDHE-ECDSA-AES128-GCM-SHA256、ECDHE-ECDSA-CHACHA20-POLY1305) 、

DTLS 1.3 (TLS_AES_128_GCM_SHA256、TLS_CHACHA20_POLY1305_SHA256)

- 每个方向断言: 握手成功、协商套件一致、双向应用数据一致 (共 8 用例)

构建与运行:

```
# 1) 构建 wolfSSL (5.9.2, 开启 DTLS1.3; 关闭 MLKEM 以使用标准 X25519/P-256 key_share)
cmake -S wolfssl-src -B wolfssl-build -DWOLFSSL_DTLS=yes -DWOLFSSL_TLS13=yes \
  -DWOLFSSL_DTLS13=yes -DWOLFSSL_MLKEM=no -DWOLFSSL_CURVE25519=yes \
  -DWOLFSSL_EXAMPLES=no -DWOLFSSL_CRYPT_TESTS=no -DBUILD_SHARED_LIBS=OFF
cmake --build wolfssl-build && cmake --install wolfssl-build --prefix wolfssl-install
```

```
# 2) 配置 jpssl (注入 wolfSSL 安装前缀与测试证书目录)
cmake -S . -B build-win -DJP_WOLFSSL_PREFIX=$PWD/wolfssl-install \
  -DJP_WOLFSSL_CERT_DIR=$PWD/wolfssl-src/certs
```

```
# 3) 运行
./build-win/tests/test_dtls_wolfssl_interop
```

调试时可设置 **WOLF_CASE=<0..7>** 只运行单个方向用例, 便于逐用例抓包定位。

TLS 稳定性测试

tests/test_tls_stability.cpp (CTest 目标 test_tls_stability) 在单次运行内反复执行:

- TLS 1.3 (Ed25519 / ECDSA P-256 / RSA-2048 证书轮换)
- TLS 1.2 (RSA)
- 0-RTT / PSK 会话恢复

- TLS-over-TCP socket 端到端握手与分块数据传输

并统计各阶段耗时、失败次数与进程内存增长（泄漏启发式检测）。

```
# 默认参数 (约 10s, 适合 CTest)
ctest --test-dir build-win -R test_tls_stability --output-on-failure
```

```
# 长稳压测: 2000 轮 TLS 1.3 + 4 个并发 worker, 每 worker 100 轮 socket 握手
./build-win/tests/test_tls_stability --iters 2000 --socket-iters 100 --threads 4
```

迭代次数可用 `--iters` / `--tls12-iters` / `--psk-iters` / `--socket-iters` / `--threads` / `--leak-mb` 等参数控制，也支持 `JPSSL_STRESS_*` 环境变量。

主测试程序

`jpssl-test` 是构建产出的主测试程序（`src/main.cpp`），覆盖各算法模块的冒烟验证：

```
LD_LIBRARY_PATH=/usr/local/musa/lib ./jpssl-test
```

持续集成

GitHub Actions 共 7 个工作流，覆盖桌面、移动与嵌入式平台；除 Linux / Windows 运行全量 `ctest` 外，iOS / macOS / Android / HarmonyOS 均为构建 + 产物校验。

工作流	平台	内容
<code>ci.yml</code>	x86_64 Linux	Release 构建 + 全量 <code>ctest</code> ； benchmarks 编译验证
<code>arm-linux.yml</code>	ARM64 Linux	<code>ubuntu-24.04-arm</code> 原生构建 + 全量 <code>ctest</code> ； <code>aarch64-linux-gnu</code> 交叉编译 + 产物校验
<code>windows.yml</code>	Windows x64 MSVC	Release 构建 + <code>ctest</code> （OpenSSL 可选， 未安装时对比测试自动跳过）； benchmarks 编译验证
<code>ios.yml</code>	macOS + Xcode	<code>ios/build-xcframework.sh</code> 构建 <code>JPSSL.xcframework</code> + Swift 编译冒烟
<code>macos.yml</code>	macOS arm64（Apple Silicon）	<code>macos/build.sh</code> 构建 <code>dylib/.so/.a</code> + <code>JPSSlC.framework</code> ， <code>lipo/codesign</code> 校验 + C 冒烟测试
<code>android.yml</code>	ubuntu + Android SDK	JDK 17 + Gradle 8.5 + NDK r26b， <code>assembleRelease</code> 产出 AAR，校验 <code>jni/arm64-v8a/libjpssl.so</code>
<code>ohos.yml</code>	ubuntu + OpenHarmony 6.1 SDK	<code>cmake/toolchains/ohos.cmake</code> 交叉编 译 <code>arm64-v8a</code> ，校验 <code>libjpssl_cpu.a/.so</code>

各平台构建命令与产物详见 平台构建（iOS / macOS / Android / HarmonyOS）。

另有 `.gitlab-ci.yml` 支持 GitLab CI。

ECDSA AVX2 / AVX512 可行性探索

范围：P-256 / P-384 / P-521 ([src/ecdsa.cpp](#))，重点是 P-256 (TLS 握手与证书链中最常用)。
结论先行：**AVX2 可行且推荐**（作为批量验证/多缓冲加速层，本机可测）；**AVX512 技术上可行但属后续可选阶段**（本机 Raptor Lake 桌面无 AVX512，无法本地验证）。无论哪条 SIMD 路线，**第一步都必须是标量算法升级**——当前实现的瓶颈不是缺 SIMD，而是基础算法（逐比特归约 + 无窗口 double-and-add）。

1. 现状与基线

当前 [ecdsa.cpp](#) 为通用 `uint256` ($4 \times u64$) + Jacobian 坐标：

- `mm_mul` 走 512 位全乘 + `mod_reduce_512` 逐比特移位比较减法归约（512 轮）；
- 标量乘是无窗口、逐 bit 的 left-to-right double-and-add（256 次倍点 + 约 128 次点加）；
- 验签是两个独立标量乘 ($u1 \cdot G$ 、 $u2 \cdot Q$) 再相加，未用 Shamir 双标量；
- `G` 无预计算表。

本机实测 (i7-13700K, MSVC Release /O2, 64 字节消息, jpssl vs OpenSSL 4.x)：

曲线	jpssl keygen	jpssl sign	jpssl verify	OSSL keygen	OSSL sign	OSSL verify	verify 差距
P-256	15.9 ms	17.5 ms	31.4 ms	5.9 μ s	14.2 μ s	40.8 μ s	约 770 $\times$
P-384	49.7 ms	55.3 ms	98.6 ms	505 μ s	527 μ s	462 μ s	约 213 $\times$
P-521	5.5 ms	19.3 ms	24.2 ms	1068 μ s	1126 μ s	930 μ s	约 26 $\times$

（OpenSSL 的 P-256 走 `ecp_nistz256` 专用汇编；P-384/P-521 走通用路径，所以差距相对小。交叉验证：jpssl 可正确验证 OpenSSL 产生的签名，`test_ecdsa` 全过。）

结论：**先做标量升级就能拿到 2~3 个数量级的收益**，与 SIMD 无关。本仓库 SM2 已经做过同样的事（Montgomery CIOS + wNAF-5 + Shamir + 批量仿射化，提速 84x/83x/135x，见 [src/sm2.cpp](#)），P-256 可直接移植该模式。

2. ECDSA 为什么适合（又为什么难）做 SIMD

适合：

- ECDSA 验签/签名的核心是**多个相互独立的 256 位标量乘**，天然适合“多缓冲”（multi-buffer）或“多通道”（lane-parallel）：AVX2 一个 YMM 寄存器同时算 4 个元素，AVX512 一个 ZMM 同时算 8 个。
- 批量验证（一次验 N 个签名）时，还有算法级共享：所有签名共享同一条倍点链（多标量乘法 MSM）。

难点：

- P-256 素数 $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$ 不是 Mersenne 类素数，SIMD 下需要专用折叠归约或 Montgomery 归约，不能像 Ed25519/Ed448 那样简单折回；
- 单个标量乘内进位链长、依赖强，**编译器自动向量化基本无望**，必须手写内建函数；
- 签名路径有秘密标量，SIMD 点运算必须保持常数时间（无秘密相关的分支/表索引）；
- 单个操作（不批量）的并行度有限，SIMD 单签收益远小于批量收益。

3. 三条技术路线

路线 A：标量算法升级（先决条件，无 SIMD）

把 `sm2.cpp` 的模式移植到 P-256（通用曲线 P-384/P-521 同样受益）：

- Montgomery 乘法（CIOS，现有 `mont_mul/mont_reduce` 可直接改参数化）；
- wNAF-5 窗口标量乘；**G** 的奇倍点表**全局预计算**（仿射表 + 混合加法）；
- 验签用 Shamir 双标量同时乘（共享倍点），替代两个独立标量乘；
- 批量仿射化（一次求逆换 N 个逆），为批量验证做准备；
- 顺手修复签名路径的常数时间问题（当前逐 bit 分支依赖秘密标量）。

预期：P-256 verify 从 ~31 ms 降到亚 0.2 ms 量级（参考 SM2 的经验量级），且所有曲线、所有平台通用。

路线 B：批量验证 `ecdsa_p256_batch_verify`（MSM，与指令集无关）

算法：Bellare–Garay–Rabin 小指数测试。给每条签名取随机权重 t_i （128 位），一次检查

$$\sum t_i \cdot (u1_i \cdot G + u2_i \cdot Q_i - R_i) == O$$

成立则全部接受；任一条无效时以 $\geq 1 - 2^{-128}$ 的概率拒绝。实现为一条共享倍点链的 MSM：

- 单个标量 $\sum t_i \cdot u1_i \bmod n$ 乘固定基点 **G**（可用预计算表）；
- N 个 $t_i \cdot u2_i \bmod n$ 乘各自的 Q_i ，加负的 $t_i \cdot R_i$ ；
- 逆元用批量求逆（1 次求逆 + N 次乘法）；
- 分块大小参照 `ed25519_batch`（128 条/块）。

收益：批量吞吐约为逐条验证的 2~4 倍（块越大越接近理论共享收益）。**指令集无关**——本仓库 Ed25519 批验证的结论就是“多标量批验证对指令集不敏感”，AVX512 后端最终只是保留兼容 API。该路线同样直接适用于 P-384/P-521。

路线 C：SIMD 字段层（沿用 `fe_448_simd.hpp` 模式）

C1：AVX2 4 通道，radix-2²⁶ × 10 肢体 + Montgomery

- 256 位素数用 10 个 26 位肢体表示；一次乘法肢体积 ≤ 52 位，schoolbook 列累加 ≤ $10 \times 2^{52} < 2^{56}$ ，**64 位 lane 内无进位链**，进位传播全部可向量化（移位 + 掩码）；
- 每个 YMM 寄存器放 4 个独立元素的同一肢体（lane-sliced 布局，与 `fe_448_simd.hpp` 完全一致），寄存器压力约 10×3，可接受；
- 参考先例：ACISP 2020 (Huang–Liu–Hu–Großschädl) 用 AVX2 2-way radix-2²⁶ 实现 SM2（同为 Weierstrass 曲线）：2-way 字段运算比串行快 1.25–1.60×，Co-Z Montgomery 阶梯快 1.31×，且抗时序/SPA；Gueron–Krasnov (J. Cryptogr. Eng. 2015) 的 P-256 定点标量乘用窗口 7 + AVX2 引擎并行执行 4 个点加法；
- 本机 13700K 支持 AVX2（CPUID 实测 AVX2=1），**可直接开发与回归**。

C2：AVX512F 8 通道（同一抽象层换 `_m512i`）

- 结构不变，lane 数 4→8；需要 AVX512F + VL (Ice Lake+ / Zen 4+)；
- 本机 (Raptor Lake 桌面) **AVX512 全系被固件熔断**（CPUID 实测 AVX512F=0，XCR0 无 opmask/ZMM），无法本地运行，需远程 CI 或服务验证。

C3: AVX512-IFMA, radix-2⁵²/2⁵¹ × 5 肢体 (vpmadd52luq/huq)

- 两条 IFMA 指令直接把 52×52→104 位乘积的低/高 52 位累加进 64 位 lane，单次字段乘的指令数最少（5×5 schoolbook = 25 luq + 25 huq）；
- Intel IPP Crypto Multi-buffer ([intel/ipp-crypto](#) 的 crypto_mb)：
[mbx_nistp256_ecdsa_sign_mb8](#) 等，AVX512-IFMA 8 缓冲 ECDSA/ECDH (P-256/384/521)，已用于 QAT 引擎的软件加速；
- [asmcrypto](#) (Rust, Zen 5 实测)：8-lane 批量 ECDSA 恢复约 15.6 μs/lane，比 8 次串行标量恢复（52.6 μs/lane）快约 3.4×，比逐条调优的 libsecp256k1 快约 1.4×；
- curve25519-dalek 的 IFMA 笔记给出完整 radix-2⁵¹ 乘/平方/归约推导。
- 成熟先例：
- 工具链：**MSVC 14.51 已实测可编译 IFMA 内建函数** ([/arch:AVX512](#) 下 [_mm512_madd52lo/hi_epu64](#) 编译通过)；GCC/Clang 长期支持。
- 硬件面：Intel Ice Lake / Tiger Lake / Rocket Lake / Sapphire Rapids, AMD Zen 4+ (Zen 4 的 512 位按 2×256 执行、无明显降频；早期 Cannon Lake 半速执行 512 位 IFMA)。

路线对比

路线	收益量级	成本/风险	平台面	本机可测
A 标量升级	verify 31 ms → <0.2 ms (数十倍)	低；代码量大但模式现成 (sm2)	全平台	是
B 批量 MSM	批量吞吐 2–4×	低；需随机数/常数时间权重	全平台	是
C1 AVX2 4-lane	批处理再 2–3× (单签为辅)	中；字段层 + 点运算重写	AVX2 (2013 年后几乎所有 x86)	是
C2 AVX512F 8-lane	C1 的约 1.5–1.8×	中；双份寄存器宽度代码	Ice Lake+ / Zen 4+	否
C3 AVX512-IFMA	8-lane 多缓冲约 3.4×/通道 (对照串行)	高；52 位冗余表示 + 多缓冲框架	需 IFMA (Ice Lake+ / Zen 4+)	否

4. 与现有代码的契合点

- 字段层抽象：[src/fe_448_simd.hpp](#) ([jf448_v](#)/掩码/比较/混合指令的 AVX2⇔AVX512 条件编译) → 可照搬为 [fe_p256_simd.hpp](#)；
- 分发机制：[include/cpu_features.hpp](#) (AVX2/AVX512/ADX 运行时检测) + [ed448_batch_dispatch.cpp](#) 的 “AVX512 > AVX2 > 标量” 优先级 + [CMakeLists.txt](#) 按源文件设 [/arch:AVX2](#)、[/arch:AVX512](#)；
- 批量验证模板：[ed25519_batch](#) (随机盲化 MSM、128 条/块、批量求逆) 与 [ed448_batch](#) (SIMD 后端 + 分发) 可分别提供 “算法层” 和 “SIMD 层” 两个样板；
- 标量内核：[src/sm2.cpp](#) 的 CIOS Montgomery、wNAF-5、Shamir、批量仿射是 P-256 的直接模板 (SM2 与 P-256 同构，仅素数不同)。

5. 风险与注意事项

- **不要给慢内核叠 SIMD**: OpenSSL 曾因 `ecp_nistz256.c` 中 AVX2 代码不可达/收益存疑而整体移除 (PR #12046)。SIMD 必须叠加在升级后的标量内核上, 否则只是“给逐比特归约加速”。
- **签名常数时间**: 当前 sign 的逐 bit 分支已非常数时间 (侧信道风险, 与 SIMD 无关), 标量升级时应一并改成固定窗口/阶梯。
- **AVX512 部署面**: Raptor Lake/Alder Lake 桌面熔断 AVX512; 老 Xeon 的 512 位指令有降频代价; AMD Zen 4+ 无此问题但按 2×256 执行。C2/C3 代码在本机无法运行, 必须靠 CI 或服务器回归。
- **编译器不会替你向量化进位链**: 全部走手写内建函数 (现有 `fe_448_simd.hpp`、`rsa_batch_avx512.cpp` 风格)。
- **P-384/P-521**: 路线 B (批量 MSM) 直接适用; 路线 C 的 $\text{radix-}2^{26}$ 需要 15/21 个肢体, 寄存器压力大、收益递减, 建议优先级放低。
- **维护成本**: 每一档 ISA 都是一份字段层 + 点运算代码, 建议用模板/宏复用点公式 (如 `ed448_simd_body.inc` 的 include 复用方式), 避免三份点公式漂移。

6. 建议路线图

阶段	内容	目标	验证
0	新增 <code>benchmarks/bench_ecdsa_ossl.cpp</code> (对标 <code>bench_ed25519_ossl</code>)	建立 jpsl vs OpenSSL 基线	□ 本机
1	标量升级: Montgomery CIOs + wNAF-5 + G 定点表 + Shamir + 修 CT	P-256 verify < 0.2 ms	□ 本机 + <code>test_ecdsa</code>
2	<code>ecdsa_p256_batch_verify</code> (MSM、随机权重、批量求逆、128/块)	批量吞吐 2–4×	本机
3	<code>fe_p256_simd.hpp</code> AVX2 4-lane + 批量/多缓冲后端 + 分发	批量再 2–3×	本机 (AVX2)
4	AVX512F 8-lane 变体; 可选 AVX512-IFMA 多缓冲 (面向 Ice Lake+/Zen4+ 服务器)	8-lane 吞吐	远程 CI/服务器

Phase 1 实测结果 (i7-13700K, MSVC Release, `benchmarks/bench_ecdsa_ossl.exe`)

曲线	操作	旧实现	新实现	提升	OpenSSL 4.x	备注
P-256	keygen	15.9 ms	170 μ s	约 93×	5.9 μ s	nistz256 专用汇编
P-256	sign	17.5 ms	190 μ s	约 92×	14.1 μ s	固定 4-bit 窗口 CT 路径
P-256	verify	31.4 ms	193 μ s	约 163×	40.6 μ s	达标 (<0.2 ms)
P-384	keygen	49.7 ms	509 μ s	约 98×	505 μ s	与 OpenSSL 持平

曲线	操作	旧实现	新实现	提升	OpenSSL 4.x	备注
P-384	sign	55.3 ms	571 μ s	约 97 $\times$	527 μ s	与 OpenSSL 持平
P-384	verify	98.6 ms	611 μ s	约 161 $\times$	465 μ s	与 OpenSSL 持平
P-521	keygen	5.5 ms	973 μ s	约 5.7 $\times$	1071 μ s	略快于 OpenSSL
P-521	sign	19.3 ms	1093 μ s	约 18 $\times$	1127 μ s	略快于 OpenSSL
P-521	verify	24.2 ms	1166 μ s	约 21 $\times$	927 μ s	接近 OpenSSL

- 三条曲线与 OpenSSL 双向互操作验证通过 (test_ecdsa 17 项全过; TLS 136 / X.509 60 / CT 95 / OpenSSL 对比 22 项全过)。
- 实现要点: 模板化 $\text{bn}\langle N \rangle$ ($N=4/6/9$) + Montgomery CIOS ($R=2^{(64N)}$); 验签 wNAF-5 + Shamir 共享倍点链; 签名/密钥生成固定 4-bit 窗口 + 常数时间表选择 (窗口数固定、无秘密索引, 仅 $H=0$ 例外分支为 $\sim 2^{-N}$ 概率的残余数据相关分支, 与 sm2.cpp 一致); G 的 1..15 倍仿射表懒预计算; 验签末尾用射影比较 $X == r \cdot Z^2$ 免去一次 Fermat 求逆。
- 说明: P-256 仍落后 OpenSSL (nistz256 为专用归约汇编 + ADX), 进一步缩小差距需手写 4-limb CIOS/FIOS 微优化或汇编, 属后续阶段; P-384/P-521 已与 OpenSSL 通用路径持平。

Phase 1.5: P-256 特殊形式归约 (nistz256 风格, C++ 版)

给 P-256 的素数域换上 Gueron-Krasnov / nistz256 特殊形式 Montgomery 归约 (p256_reduce_special, 常数 0xffffffff00000001, 4 轮低字折叠 + 条件减 p, 参照 Go p256_asm_amd64.s; 经 1 万组随机输入与通用 Montgomery 对照验证)。mod n (群阶) 仍走通用 CIOS。

曲线	操作	Phase 1	Phase 1.5	提升	OpenSSL 4.x
P-256	keygen	169.5 μ s	97.4 μ s	1.74 $\times$	5.9 μ s
P-256	sign	190.2 μ s	118.3 μ s	1.61 $\times$	14.1 μ s
P-256	verify	194.8 μ s	116.2 μ s	1.68 $\times$	40.6 μ s

verify 的时间分布 (实测): wNAF 双标量 (256 倍点 + ~ 170 混合加) 约 84 μ s、mod-n 求逆约 17 μ s、Q 的奇倍表约 14 μ s、哈希约 0.4 μ s。

与 OpenSSL 差距的归因: 当前 C++ 域乘 (16 次 umul128 + 折叠) 约 22 ns ($\approx 75\text{--}100$ 周期), nistz256 汇编 (MULX/ADCX/ADOX + 操作数扫描乘积) 约 15–20 周期; 点公式 (dbl-2001-b / add-2007-bl) 双方相同。因此主要差距在域运算速度, 若要追平 OpenSSL 需要 MASM ADX 域核心 (仓库已有先例 fe51_mul_adx.asm / rsa_mont_asm_win.asm), 预期 verify 可到 $\sim 45\text{--}55$ μ s。这属于下一阶段 (Phase 1.6), 工作量约半个到一个工作日。

Phase 1.6: MASM ADX 域核心 (已完成)

新增 `src/ecdsa_p256_adx.asm`: `jpssl_p256_mul_adx`, 结构来自 OpenSSL `ecp_nistz256-x86_64.pl` 的 `__ecp_nistz256_mul_montx` (交错 CIOS + 特殊归约, MULX/SHLX/SHRX + ADCX/ADOX 双进位链), 按 Windows x64 寄存器重映射。
`mont_mul` 在 `M.special && cpu_has_adx()` 时走汇编, 否则回退 C++ 特殊归约;
GCC/Clang 平台不受影响 (仍用 C++ 路径)。2 万组随机输入与通用 Montgomery 对拍通过。

P-256	Phase 1.5	Phase 1.6 (ADX asm)	OpenSSL 4.x	差距
keygen	97.4 μs	56.0 μs	5.9 μs	9.4×
sign	118.3 μs	77.0 μs	14.1 μs	5.5×
verify	116.2 μs	63.4 μs	40.7 μs	1.56×

相对原始实现: verify 31.4 ms → 63.4 μs (约 495×)、keygen 15.9 ms → 56 μs (约 284×)、sign 17.5 ms → 77 μs (约 227×)。域乘 ~8.4 ns (约 35–40 周期, 已与 nistz256 同量级), jac_dbl 153 ns。

残余差距归因: 1) `jac_dbl` 实测 153 ns vs 8 次域乘理论 ~67 ns——MSVC 对点公式 (多临时变量) 的代码生成有 ~2 倍开销, 点运算本身写成汇编可再砍一半; 2) `mod n` 求逆仍走通用 CIOS (约 27 ns/次, 未用特殊归约); 3) 签名/密钥生成的固定 4-bit 窗口 (64 次加法) 不如 nistz256 的 8-bit 固定基表。若三者都做, verify 可望接近 40 μs。

Phase 1.7: 三项收尾优化 (已完成)

- 1. **mod n (群阶) ADX 归约:** `jpssl_p256_ord_mul_adx` (转写 `ecp_nistz256_ord_mul_montx`, `K0 = 0xccd1c8aaee00bc4f + n` 的 4 字), `mod n` 域乘 ~27 → 11.6 ns, `mod_inv n 17` → 6.2 μs;
- 2. **点运算汇编:** `jpssl_p256_dbl` (`dbl-2001-b`, 9 次域乘 + 模加减全调度在栈帧/寄存器) 与 `jpssl_p256_madd` (混合加, `P` 无穷远走固定模式分支、`Q` 无穷远走无分支掩码、`H=0` 走可忽略分支), `dbl 155` → 140 ns、`madd 157` → 149 ns;
- 3. **固定基点窗口组合 (comb):** 预计算 64×15 个仿射点 `d·2^(4k)·G` (61KB, 懒构建), 热点路径零倍点、64 次混合加 + CT 选点, `keygen/sign` 的固定基部分约 48 → 10 μs。

最终 (i7-13700K, MSVC Release, 与 OpenSSL 4.x 同机对比) :

P-256	原始	最终	累计提升	OpenSSL 4.x	差距
keygen	15.9 ms	15.9 μs	约 1000×	5.9 μs	2.7×
sign	17.5 ms	23.1 μs	约 758×	14.1 μs	1.6×
verify	31.4 ms	51.1 μs	约 614×	40.8 μs	1.25×

全套测试 (`test_ecdsa` 17 项、X.509 60、CT 95、`ossl_verify` 7/7、SM、OpenSSL 对比 22) 通过, 双向互操作 PASS。残余差距主要是域乘仍未内联进点公式 (每点运算 ~10 次函数调用), 以及 nistz256 的定点表更大 (本实现为 CT 选点约束所限)。

Phase 2: ECDSA 接入 TLS 密钥交换 (已完成)

TLS 的 ECDSA 此前只用于证书签名/握手认证 (0x0403/0503/0603, `CertificateVerify`), ECDHE 密钥交换仅支持 X25519/X448/curveSM2。本轮补齐:

- **ECDHE 共享密钥 API:** `ecdsa_p256_ecdh` / `ecdsa_p384_ecdh`
(`shared = X(priv·pub)`, [RFC 8446](#) §4.2.8.2 定义; 与 OpenSSL `ECDH_compute_key` 对拍通过) ;
- **命名组:** `NamedGroup::secp256r1(0x0017)` / `secp384r1(0x0018)`, 客户端 `tls_session.ks_group` 设为对应组即启用; `key_share` 采用 [RFC 8446](#) 裸 `x||y` 格式
(P-256 64B / P-384 96B, 无 0x04 前缀) ;
- **服务端协商:** 优先级 SM2 > secp256r1 > secp384r1 > X448 > X25519;
- **测试:** TLS 1.3 完整握手 (P-521 证书 + P-256/P-384 ECDHE) 新增 20 项断言,
`test_tls 136` → **156 项全过**; `test_ecdsa` 新增 ECDH 与 OpenSSL 对拍 (P-256/P-384 均 PASS) 。

Phase 3: ECDHE 批量/多缓冲加速 (第二种方案, 本地可验证部分已完成)

针对「ECDHE 能否做汇编加速、能快多少」的结论: 单条操作受限于 4-limb 依赖链, 汇编收益约 25–30%; 真正能拉开差距的是**批量/多缓冲** (multi-buffer) ——把 N 条 ECDH 打包, 块内共享求逆。本轮按第二种方案交付:

- **批量 API:** `ecdsa_p256_ecdh_batch` / `ecdsa_p384_ecdh_batch`
(`shared/priv/pub` 连续数组; `pub` 为裸 `x||y`, 与 [RFC 8446](#) `key_share` 一致; `count` ≥ 1, 内部按 16 条一块) ;
- **实现:** 块内建 8n 个奇倍点表 (Jacobian, 仅 `madd` 不求逆) → `batch_affine_many` 整批 1 次 Fermat 求逆
→ 各自 `wNAF-5` → 输出再整批 1 次求逆。每 16 条仅 2 次求逆 (逐条需 32 次) ; `Z=0` 的异常点逐点回退保持正确性;
- **验证:** `test_ecdsa Test 19` (批量 vs 逐条对拍 N=17 覆盖尾部块, P-256/P-384 全 PASS, 批量[0] 与 OpenSSL `ECDH_compute_key` 一致) ; `bench_ecdh_batch` 逐字节核对 PASS。

实测 (i7-13700K, MSVC Release /O2, `benchmarks/bench_ecdh_batch.exe`) :

曲线	模式	N=100 ns/op	N=1000 ns/op	vs 逐条	vs OpenSSL
P-256	逐条	55.4 μs	54.6 μs	1.00×	0.55×
P-256	批量	46.3 μs	46.2 μs	1.20×	0.66×
P-384	逐条	552.7 μs	560.8 μs	1.00×	0.92×
P-384	批量	436.7 μs	439.1 μs	1.27×	1.16×

(OpenSSL 同机同源参考: P-256 ≈ 30.6 μs、P-384 ≈ 507 μs。)

关于"逐条 55 μs 比之前 42.8 μs 慢"的说明: `per-op` 代码路径未变, 差异来自测量对象。

固定同一把密钥实测 42.5 μs (与旧基线一致) ; 轮转 100/1000 把不同密钥时, `wNAF` 主循环的数据相关分支

(`wnaf5` 奇偶测试、`d>0/d<0`、`pt_madd` 内部分支) 无法被分支预测器跟踪, 每次调用多付约 11 μs 惩罚:
K=2 轮转 43.0 μs, K=16 即跳到 53.2 μs, K≥256 稳定在 54.7–55.2 μs;

同一批密钥逐把连续测 64 次平均 43.2 μs。批量 vs 逐条 (1.20×) 是公平对比 (两边都轮转不同密钥) ;
真实多变密钥场景下进一步提速应把 `wNAF` 换成固定窗口 + 常数时间选点 (无数据分支) , 可同时消除该惩罚与时序侧信道。

结论与后续: 纯标量批量已摊薄每条 2 次求逆 (约 9 μs/条) , P-256 约 1.20×、P-384 约 1.27× 吞吐提升,
P-384 批量已反超 OpenSSL。批量收益目前受限于 `wNAF` 主循环本身仍为串行标量;
再上 2–4× 需叠 AVX2 4-lane / AVX512-IFMA 8-lane 字段内核 (路线 C) ,

其中 AVX512-IFMA 多缓冲约 3.4×/lane (对照 IPP crypto_mb 与 asmcrypto 数据) , 本机 Raptor Lake 无 AVX512, 该部分留待远程 CI/服务器验证。

Phase 3.1: 单次 ECDH 加速——点运算汇编无分支化 (已实现)

上节发现"逐条 55 μs vs 固定密钥 42.8 μs"的分支惩罚后, 进一步定位到根因: `jpssl_p256_dbl / jpssl_p256_madd` 汇编里的特殊归约用了数据相关条件跳转 (`cmp/ja/jb/jne` 的"比较后决定是否减 p", 每个 db1 约 6 个归约块、每个 madd 约 5 个)。密钥固定时分支结果恒定、预测全中 (42.5 μs) ; 每次换密钥时全部猜错 (54.7 μs, +12 μs) 。

修复: 把 15 个 db1 归约块 + 2 个宏 + 7 个 madd 内联块全部改成无分支:

- 减法归约: `setc` 保存借位 → `neg` 重建 CF → 4 条 `cmovc` 选择 r 或 r+p;
- 加法归约: `t = r - p` 后按 `borrow & ~carry_out` 掩码加回 p (a,b<p 时 `carry_out==1` 必然 `borrow==1` , 掩码逻辑闭合) ;
- 保留 madd 的 P 无穷远 / H==0 罕见分支 (概率 ~2^-N, 恒定可预测) 。

验证: 20k 随机点 asm vs C++ 对拍 + 边界 (`dbl(inf)`、`madd(inf,A)`、`madd(A,inf)`、`madd(A,A)`、`madd(A,-A)`) + 200 组 ECDH 自检全过; `test_ecdsa` 19 项、`test_tls` 156 项全过。

实测 (i7-13700K, MSVC Release, 同会话 A/B: stash 旧 asm → 重建 → 测 → 恢复新 asm → 重建 → 测) :

场景	旧 (分支版)	新 (无分支版)	变化
ECDH 固定同一密钥循环 (微基准)	42.5 μs	48.7 μs	+15% (掩码指令开销 > 完美预测分支)
ECDH 完全随机密钥 (微基准 K≥256)	54.7 μs	49.6 μs	-9%
ECDH 轮转 100 把密钥 (<code>bench_ecdh_batch</code>)	49.4 μs	49.3 μs	≈持平
ECDH 批量 16 条/块	41.2 μs	41.0 μs	≈持平
ECDSA verify (固定签名微基准)	57.6 μs	58.5 μs	+1.5%
ECDSA keygen / sign	14.9 / 22.2 μs	14.8 / 22.0 μs	≈持平

结论: 吞吐在标准基准上基本持平; 真实随机密钥场景略快 (-9%) , 固定密钥微基准略慢 (+15%) —— 差异只取决于分支预测器能否学会当前密钥的模式 (真实 TLS 每握手都是新密钥, 学不会) 。主要收益是常数时间: 点运算不再有密钥相关的条件跳转, keygen/sign (已用 CT comb) 成为整体常数时间 , 消除了一类时序侧信道。P-384/P-521 走 C++ 点运算 (`mod_add/mod_sub` 仍有分支) , 不受此改动影响。剩余的单次提速空间: 1) 求逆加法链/汇编 (`mod_inv p` 现 4.5 μs, 2 次/操作 ≈ 9 μs, 专用链可省 3-4 μs) ; 2) 点运算主循环进一步去栈往返/提升 ILP (`wNAF` 31 μs, 理论域乘下限 ~23 μs) ; 3) 固定窗口 CT 查表 (实测 `ct_fixed_window` 因 `select` 开销反慢, 需先优化 `select` 才有意义) 。

Phase 3.2: P-256 专用求逆加法链 + 汇编 (已实现)

完成上一节剩余空间第 1 项: `mod_inv p` 从通用 Fermat ($256 \text{ sq} + 128 \text{ mul}$, $4.27 \mu\text{s}$) 换成 addchain 专用链 **`255 sq + 12 mul`** (与 Go `crypto/internal/nistec/flat/p256_invert.go` 同源, addchain v0.4.0 生成, 指数 $p-2$)。

实现: `src/ecdsa_p256_adx.asm` 新增 `jpssl_p256_inv_adx(rcx=r, rdx=a)`, 栈上放 14 个 4-limb 临时变量, 267 步全部调用现有 `jpssl_p256_mul_adx` (输出已完全归约, 无需额外约减); 开头把输入复制到栈上, 允许 `r==a` 别名; 0 输入自然得到 0 (与 `mont_pow` 语义一致)。
mod_inv 分发: 仅 `N==4 && M.special==1 && g_p256_adx_ok` 走新路径; P-256 阶 `n` (`special==2`, ECDSA 签名用) 与 P-384/P-521 仍走通用 Fermat。

验证: 20k 随机值与 `mont_pow` (同一 `mul_adx` 后端) 对拍全过; 性质 `mont_mul(a, inv(a))==R` 全过; 边界 `inv(1)`、`inv(mont 1)`、`inv(p-1)`、`inv(2)`、`inv(0)`、`r==a` 别名全过; `test_ecdsa` 19 项、`test_tls` 156 项全过。常数时间: 整条链为固定指令序列、无数据相关分支。

实测 (i7-13700K, MSVC Release, 同进程 A/B) :

指标	Fermat (旧)	addchain (新)	变化
<code>mod_inv p</code> (直接调用)	$4.27 \mu\text{s}$	$2.89 \mu\text{s}$	-32% ($1.47\times$)
ECDH per-op 固定密钥	$48.7 \mu\text{s}$	$46.7 \mu\text{s}$	-2 μs
ECDH per-op 批量 16	$41.0 \mu\text{s}$	$41.2 \mu\text{s}$	~持平 (噪声)

求逆理论下限约 $2.3 \mu\text{s}$ ($267 \times 8.7 \text{ ns}$ 域乘); 差距来自每次 `mul_adx` 的 `call/push/pop` 开销, 后续可做"轻量 `mul`" (免寄存器保存) 或把链内平方内联到主循环。

Phase 3.3: 合并上游后的 flaky 堆损坏排查 (已解决)

合并上游 TLS socket 增强 (9d88f7e) 后 `test_tls_socket` 约 50% 概率堆损坏崩溃 (0xC0000374)。
排查结论有两层:

- 代码层根因** (`include/tls_socket.hpp` 的 `tls_co_task`) : `await_resume()` 销毁内层协程帧后未把 `h_` 置空; `co_await` 嵌套任务(...) 全表达式结束时临时任务对象析构会再次 `destroy()` 同一帧 $\rightarrow$ double-free。修复: 销毁后 `h_ = nullptr` (模板与 void 特化两处)。用最小无 socket 复现 (`scratch_co_task_probe`, 纯嵌套 `co_await` 20 万次) 验证修复语义。
- 构建层陷阱:** `ninja` 的依赖跟踪在合并/改头文件后没有重编 `tls_socket.cpp.obj` (同样曾导致 `test_tls.cpp.obj` 按旧 `tls_session` 布局编译), 旧协程代码与新头文件混链 $\rightarrow$ 修复"看似无效"。
`ninja -t clean` 全量重建后, `test_tls_socket` 连续 43 次全过, `test_ecdsa/test_tls` 全过。
若后续改头文件出现"没重编"现象, 优先怀疑依赖跟踪 (需要 `clean` 重建或检查 `ninja deps`)。

7. 参考资料

- Bellare, Garay, Rabin: [Fast Batch Verification for Modular Exponentiation and Digital Signatures](#) (批量验证理论, 小指数测试)
- Gueron, Krasnov: [Fast prime field elliptic-curve cryptography with 256-bit primes](#) (P-256, 定点乘窗口 7 + AVX2 并行 4 点加, J. Cryptogr. Eng. 5(2), 2015)
- Huang, Liu, Hu, Großschädl: [Parallel Implementation of SM2 Elliptic Curve Cryptography on Intel Processors with AVX2](#) (ACISP 2020: radix-2²⁶ 2-way AVX2 字段算术, Co-Z 阶梯 1.31 $\times$, 抗时序)

- Intel IPP Crypto: Multi-buffer Cryptography ([mbx_nistp256_ecdsa_sign](#), [IPP-Crypto 源码](#))
- curve25519-dalek: [AVX512-IFMA 笔记](#) (radix-2⁵²/2⁵¹ 乘法的完整推导)
- asmcrypto: [AVX-512 批量 ECDSA 恢复基准](#) (8-lane, 约 15.6 μs/lane, Zen 5)
- OpenJDK: [P-256 Montgomery 乘 AVX2 内建函数 \(约 60–80% 提升\)](#)
- OpenSSL: [移除 ecp_nistz256 中不可达的 AVX2 代码 \(PR #12046\)](#)